

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CARRERA DE CIENCIAS DE LA COMPUTACIÓN



**DETECCIÓN Y COMPENSACIÓN
INTEGRADA DE FALLAS EN UAV
CUADRICÓPTERO MEDIANTE ESTIMACIÓN
DE ESTADOS Y APRENDIZAJE TEMPORAL
LIGERO**

TRABAJO DE INVESTIGACIÓN

Para optar el grado de Bachiller en Ciencia de la Computación

AUTORA

Yacira Nicol Campoverde San Martín 

ASESOR

Victor Flores Benites 

Lima - Perú

2026

Dedicatoria:

*A mis padres, por enseñarme que el esfuerzo siempre encuentra su camino y por
ser mi mayor motivación para seguir adelante y alcanzar mis sueños.*

Agradecimientos:

Agradezco a Dios, por brindarme la fortaleza necesaria para culminar esta etapa del trabajo; a mi familia, por su apoyo permanente e incondicional durante todo el proceso. Gracias a mi asesor, por su orientación, paciencia y valiosas contribuciones al desarrollo de este trabajo de investigación.

TABLA DE CONTENIDO

	Pág.
RESUMEN	1
ABSTRACT	3
INTRODUCCIÓN	5
Presentación del tema de investigación	5
Descripción de la situación problemática	7
Formulación del problema	8
Objetivos de investigación	9
Justificación	10
Alcance y limitaciones / restricciones	11
 CAPÍTULO I REVISIÓN CRÍTICA DE LA LITERATURA	 12
1.1 Métodos Basados en Modelos y Observadores de Estado	12
1.2 Detección de Fallas Basada en Aprendizaje Profundo	16
1.3 Control Tolerante a Fallas Basado en Aprendizaje por Refuerzo	17
1.4 Estrategias Híbridas de Adaptación en Tiempo Real	20
1.5 Consideraciones finales	22
 CAPÍTULO II MARCO TEÓRICO	 23
2.1 Vehículos Aéreos No Tripulados tipo Cuadricóptero	23
2.1.1 Arquitectura y Principio de Funcionamiento	24
2.1.2 Modelo Dinámico	25

2.2	Fallas en Sistemas UAV Cuadricóptero	26
2.2.1	Clasificación de Fallas	27
2.2.2	Escenarios de Fallas Simultáneas sobre la Dinámica del Vuelo	28
2.3	Arquitecturas de Aprendizaje Profundo para Control Secuencial	28
2.3.1	Redes de Memoria de Largo y Corto Plazo (LSTM)	28
2.3.2	Modelos de Espacio de Estados Selectivos: Mamba SSM	31
2.4	Aprendizaje por Imitación para Políticas de Control	33
2.4.1	Behavioral Cloning	33
2.4.2	DAgger	34
2.4.3	Aprendizaje por Refuerzo Inverso	35
2.4.4	GAIL	35
2.4.5	Comparación de Métodos	36
2.5	Arquitectura de Control Tolerante a Fallas	36
2.5.1	Arquitecturas de Control Jerárquico en FTC	37
2.6	Entorno de Simulación: gym-pybullet-drones	38
 CAPÍTULO III MARCO METODOLÓGICO		40
3.1	Descripción General de la Propuesta	41
3.2	Pipeline Metodológico Propuesto	44
3.2.1	Generación del Dataset de Imitación	46
3.2.2	Preprocesamiento y Normalización	49
3.2.3	Entrenamiento por imitación mediante clonación de comportamiento (Behavioral Cloning)	50
3.2.3.1	Proceso de entrenamiento	51
3.2.3.2	Modelo LSTM	53
3.2.3.3	Modelo Mamba SSM	53
3.2.4	Entrenamiento del Compensador PPO	55
3.2.4.1	Entorno de entrenamiento	55
3.2.4.2	Incremento progresivo de falla	56

3.2.4.3	Función de recompensa	56
3.2.4.4	Hiperparámetros del agente PPO	57
3.2.5	Evaluación de controladores	57
3.2.6	Métricas de Desempeño	60
CAPÍTULO IV RESULTADOS PRELIMINARES		63
4.1	Convergencia del Entrenamiento por Imitación	63
4.2	Evaluación en Vuelo Normal (Sin Falla de Motor 0)	64
4.3	Evaluación bajo Falla de Motor 0	69
4.3.1	Caso representativo: falla del 15 % de potencia	70
4.4	Síntesis de resultados preliminares	76
CAPÍTULO V CONCLUSIONES Y TRABAJOS FUTUROS		78
5.1	Conclusiones	78
5.2	Trabajos Futuros	80

ÍNDICE DE TABLAS

Tabla 2.1	Definición de variables en el modelo dinámico del cuadricóptero	26
Tabla 2.2	Definición de variables en el modelo LSTM	30
Tabla 2.3	Comparación de métodos de aprendizaje por imitación	36
Tabla 2.4	Parámetros físicos y de propulsión de los modelos de drones disponibles en <i>gym-pybullet-drones</i> [25].	38
Tabla 3.1	Métricas de Nivel 1 - Evaluación de la Política de Imitación . .	42
Tabla 3.2	Métricas de Nivel 2 - Evaluación de la compensación ante fallas de motor	43
Tabla 3.3	Variables de control que se mantienen constantes en todos los experimentos	43
Tabla 3.4	Definición de los <i>waypoints</i> de la trayectoria.	47
Tabla 3.5	Variables que conforman el vector de estado del UAV	48
Tabla 3.6	Características del dataset de imitación generado.	48
Tabla 3.7	Hiperparámetros de entrenamiento comunes a LSTM y Mamba SSM.	52
Tabla 3.8	Parámetros de cada bloque Mamba.	54
Tabla 3.9	Hiperparámetros del agente PPO.	58
Tabla 3.10	Trayectorias de evaluación reproducibles.	58
Tabla 3.11	Condiciones de terminación de un episodio de evaluación. . . .	59
Tabla 3.12	Métricas de desempeño y controladores a los que aplican. . . .	62

ÍNDICE DE FIGURAS

Figura 2.1	Estructura esquemática de un UAV tipo cuadricóptero y sus sistemas de coordenadas: ángulo de alabeo ($roll, \phi$), cabeceo ($pitch, \theta$) y guiñada (yaw, ψ) respecto al marco inercial. Tomado de [21].	25
Figura 3.1	Parte 1 - Generación del dataset de imitación.	44
Figura 3.2	Parte 2 - Entrenamiento por Behavioral Cloning.	45
Figura 3.3	Parte 3 - Entrenamiento del compensador PPO.	45
Figura 3.4	Parte 4 - Evaluación y comparación de desempeño de controladores	46
Figura 3.5	Waypoints secuenciales por episodio.	47
Figura 4.1	Curvas de pérdida de validación (MSE) durante el entrenamiento por imitación. LSTM: mín. 0.0058; Mamba SSM: mín. 0.0052.	64
Figura 4.2	Seguimiento de trayectoria 3D en vuelo sin fallas	65
Figura 4.3	Orientación 3D del dron	65
Figura 4.4	Seguimiento de trayectoria 3D en vuelo sin fallas	65
Figura 4.5	Orientación 3D del dron	65
Figura 4.6	Seguimiento de trayectoria 3D en vuelo sin fallas	66
Figura 4.7	Orientación 3D del dron	66
Figura 4.8	Seguimiento de trayectoria 3D en vuelo sin fallas	66
Figura 4.9	Orientación 3D del dron	66
Figura 4.10	Seguimiento de trayectoria 3D en vuelo sin fallas	67
Figura 4.11	Orientación 3D del dron	67

Figura 4.12 Orientaciones (Roll - Pitch - Yaw).	68
Figura 4.13 Comparación de métricas en vuelo normal: error de posición final, longitud de trayectoria recorrida y distancia mínima alcanzada a cada waypoint.	69
Figura 4.14 Tasa de éxito (%) bajo falla del motor 0 para cuatro niveles de pérdida de potencia.	70
Figura 4.15 Trayectoria 1 bajo una pérdida de potencia del 15 %. El con- trolador PID colisiona aproximadamente en $t = 3.69s$, LSTM en $t = 3.77s$ y Mamba en $t = 6.04s$	71
Figura 4.16 Trayectoria 2 con una pérdida de potencia del 15 %. Los tres controladores colisionan aproximadamente en $t = 3.8s$ sin recuperar la estabilidad.	72
Figura 4.17 Trayectoria 3 con una pérdida de potencia del 15 %. El contro- lador LSTM colisiona en $t = 4.50s$ y Mamba en $t = 5.06s$, registrando tiempos de vuelo superiores a los obtenidos en la trayectoria 2.	73
Figura 4.18 Trayectoria 4 con una pérdida de potencia del 15 %. Los contro- ladores PID, LSTM y Mamba colisionan aproximadamente en $t = 3.67s$	74
Figura 4.19 Trayectoria 5 bajo una pérdida de potencia del 15 %. El con- trolador LSTM colisiona en $t = 3.67s$ y Mamba en $t = 3.98s$	75

RESUMEN

Los vehículos aéreos no tripulados tipo cuadricóptero son plataformas ampliamente empleadas en tareas de inspección, monitoreo y transporte autónomo. Su operación en entornos reales los expone a fallas de actuador, como la pérdida parcial de efectividad en uno o más motores, lo que puede comprometer la estabilidad del vuelo.

Si bien el título de este trabajo contempla tanto la detección como la compensación de fallas, la presente investigación se centra en la etapa de imitación del controlador PID y compensación de fallas de un motor. La incorporación de un módulo de detección autónomo de fallas se abordará en la siguiente entrega.

En esta primera etapa se propone un pipeline de control tolerante a fallas para un cuadricóptero Crazyflie CF2X compuesto por dos componentes: un controlador base aprendido mediante clonación de comportamiento (*Behavioral Cloning*, BC) a partir de demostraciones de un controlador PID, y un compensador residual entrenado con Proximal Policy Optimization (PPO) que genera correcciones de acción cuando el sistema se desvía de la trayectoria de referencia. Las fallas de actuador son inyectadas directamente en el simulador, sin que el sistema deba identificarlas de forma autónoma.

Para la etapa de imitación se comparan dos arquitecturas: redes de memoria de largo y corto plazo (LSTM) y el modelo de espacio de estados selectivo Mamba SSM. Mamba SSM alcanzó una pérdida de validación mínima de $\mathcal{L}_{\text{val}} = 0.0052$, un 10.3 % inferior a la de la LSTM ($\mathcal{L}_{\text{val}} = 0.0058$), y completó el 100 % de las trayectorias en vuelo nominal, frente a la LSTM que no logró completar la Trayectoria 2 por acumulación de errores de covariables. Estos resultados motivaron la selección de Mamba SSM como base del compensador PPO.

La arquitectura Mamba+PPO fue evaluada ante pérdidas de potencia del 5 %, 10 % y 15 % en el motor 0 dentro del simulador *gym-pybullet-drones*. En el escenario de mayor severidad (15 %), PID y LSTM perdieron la estabilidad a los 0.69 s y 0.77 s tras la activación de la falla, y Mamba BC prolongó el vuelo 3.04 s antes de colapsar. Únicamente Mamba+PPO completó la misión con aterrizaje controlado, alcanzando una tasa de éxito del 100 % en todos los niveles evaluados.

Palabras clave: Control tolerante a fallas; Cuadricóptero; Aprendizaje por imitación; Mamba SSM; Proximal Policy Optimization; Compensación de fallas de actuador.

ABSTRACT

INTEGRATED FAULT DETECTION AND COMPENSATION IN QUADRICOPTERO UAVS USING STATE ESTIMATION AND LIGHTTIME LEARNING

Quadrotor unmanned aerial vehicles (UAVs) are widely used platforms for inspection, monitoring, and autonomous transport tasks. Their operation in real-world environments exposes them to actuator faults, such as partial loss of effectiveness in one or more motors, which can compromise flight stability.

Although the title of this work encompasses both fault detection and compensation, the present research focuses on the imitation of a PID controller and the compensation of single-motor faults. The integration of an autonomous fault detection module is deferred to the next deliverable.

In this first stage, a fault-tolerant control pipeline is proposed for a Crazyflie CF2X quadrotor consisting of two components: a base controller learned through Behavioral Cloning (BC) from demonstrations of a PID controller, and a residual compensator trained with Proximal Policy Optimization (PPO) that generates corrective actions when the system deviates from the reference trajectory. Actuator faults are injected directly into the simulator; the system is not required to identify them autonomously.

For the imitation stage, two architectures are compared: Long Short-Term Memory (LSTM) networks and the selective state-space model Mamba SSM. Mamba SSM achieved a minimum validation loss of $\mathcal{L}_{\text{val}} = 0.0052$, a 10.3 % reduction over the LSTM ($\mathcal{L}_{\text{val}} = 0.0058$), and completed 100 % of the trajectories in nominal flight, whereas the LSTM failed to complete Trajectory 2 due to covariate shift error accumulation. These results motivated the selection of Mamba SSM as the base architecture for the PPO compensator.

The Mamba+PPO architecture was evaluated under motor 0 power losses of 5 %, 10 %, and 15 % within the *gym-pybullet-drones* simulator. In the highest-severity scenario (15 %), the PID controller and LSTM lost stability at 0.69s and 0.77s after fault activation, and Mamba BC extended flight 3.04s post-fault before crashing. Only Mamba+PPO completed the mission with a controlled landing, achieving a 100 % success rate across all evaluated severity levels.

Keywords: Fault-tolerant control; Quadroter; Behavioral cloning; Mamba SSM; Proximal Policy Optimization; Actuator fault compensation.

INTRODUCCIÓN

Presentación del tema de investigación

Los vehículos aéreos no tripulados (*Unmanned Aerial Vehicles*, UAV) son aeronaves operadas de forma remota o autónoma, sin piloto a bordo, cuyo control puede realizarse mediante un operador en tierra o mediante sistemas de navegación autónoma. Desde su aparición, estos sistemas se han consolidado como una tecnología clave debido a su capacidad para ejecutar tareas de manera eficiente en entornos donde la intervención humana resulta costosa, peligrosa o de difícil acceso. Gracias a estas capacidades, actualmente se emplean en una amplia variedad de aplicaciones, entre las que se encuentran la inspección de infraestructuras, la agricultura de precisión, la vigilancia, el monitoreo ambiental, las labores de búsqueda y rescate y el transporte de productos, entre otras. Como resultado de su creciente adopción, los UAV se han convertido en un importante plataforma de investigación. En función de los distintos requerimientos operacionales que deben satisfacer, estos vehículos presentan diferentes configuraciones aerodinámicas [1, 2].

De acuerdo con su configuración aerodinámica, los UAV pueden clasificarse en aeronaves de ala fija, de ala rotatoria y de configuración híbrida [1]. Los UAV de ala fija se caracterizan por ofrecer mayor autonomía y eficiencia energética, lo que los hace adecuados para misiones de larga duración y recorridos extensos. Por su parte, los UAV de ala rotatoria, entre ellos los cuadricópteros, destacan por su capacidad de despegue y aterrizaje vertical, facilidad de maniobra y habilidad para mantenerse en vuelo estacionario, características que les permiten operar en espacios reducidos y ejecutar tareas que requieren un alto nivel de precisión y control de posición [1, 2]. Asimismo, los UAV de configuración híbrida integran las ventajas de las aeronaves de ala fija y de ala rotatoria, combinando la autonomía y la eficiencia energética con la capacidad de despegue y aterrizaje vertical [1].

Entre las diferentes configuraciones aerodinámicas de UAV, los cuadricópteros constituyen una de las plataformas más estudiadas en la literatura [1, 2]. Esto se debe a que sus características, como la capacidad de despegar y aterrizar verticalmente, mantenerse en vuelo estacionario y realizar maniobras con alta precisión, los hacen adecuados para muchas de las aplicaciones mencionadas anteriormente. Por esta razón, la presente investigación toma al cuadricóptero como plataforma de estudio.

Sin embargo, estas mismas características hacen que su operación dependa del correcto funcionamiento de los sistemas de percepción, control y propulsión. Durante una misión, pueden presentarse fallas en sensores, degradación de actuadores con pérdidas de efectividad de hasta el 36 % en un solo motor sin llegar a una falla total [3?] o perturbaciones externas, como ráfagas de viento o cambios en las condiciones del entorno, que afectan el desempeño del vehículo [1, 2, 4, 5]. Cuando estas anomalías no son detectadas y controladas en el momento correcto, pueden ocasionar errores en la estimación del estado, desviaciones en el seguimiento de la trayectoria, pérdida de estabilidad e incluso provocar daños materiales irreparables y pérdidas económicas significativas [2, 5]. Por ello, garantizar que un cuadricóptero mantenga un comportamiento estable y confiable frente a estas condiciones constituye uno de los principales desafíos en el diseño de sistemas de control para UAV.

Con el propósito de enfrentar estos desafíos, se han propuesto diversas estrategias orientadas a incrementar la confiabilidad de los UAV, entre las que destaca el Control Tolerante a Fallas (*Fault-Tolerant Control*, FTC), considerado una de las principales líneas de investigación cuyo objetivo es diseñar sistemas de control capaces de mantener un nivel aceptable de desempeño ante la presencia de fallas en los componentes del vehículo o perturbaciones externas, sin que estas comprometan la estabilidad ni el cumplimiento de la misión [1, 4]. A diferencia de los sistemas de control convencionales, que asumen condiciones de operación nominales, los sistemas FTC incorporan mecanismos de detección, diagnóstico y compensación de fallas que

permiten al vehículo adaptarse a condiciones degradadas y continuar operando de forma segura [1, 2].

Si bien los enfoques actuales resuelven parcialmente este problema los métodos basados en modelos detectan fallas mediante residuales [4, 6], las arquitecturas (LSTM, *Long Short-Term Memory*) alcanzan una precisión del 79.48 % prediciendo fallas 30 segundos antes de un evento crítico [5], y las políticas de aprendizaje por refuerzo reportan tasas de éxito del 95 % en compensación de pérdidas de actuador [3], cada enfoque aborda de forma aislada uno solo de los tres tipos de falla y no considera la viabilidad computacional en hardware embebido. Asimismo, diversos enfoques presentan limitaciones en su capacidad de generalización a diferentes escenarios de operación, lo que restringe su aplicación en entornos reales [1, 3, 5, 7].

Frente a esta brecha, el presente estudio propone emplear la arquitectura Mamba basada en modelos de espacio de estados selectivos como controlador base entrenado mediante imitación de un controlador PID, apoyado en un estimador de estados para la detección de fallas y combinado con un compensador residual basado en aprendizaje por refuerzo, con el fin de detectar y compensar de forma integrada fallas en actuadores, sensores y perturbaciones externas en un cuadricóptero Crazyflie CF2X, validado en el simulador gym-pybullet-drones bajo restricciones de viabilidad computacional para plataformas embebidas de bajo costo. Con ello, se busca contribuir al desarrollo de sistemas de control tolerante a fallas más robustos, eficientes y viables para su implementación en plataformas UAV de bajo costo.

Descripción de la situación problemática

Durante la operación de un cuadricóptero, la presencia de fallas y perturbaciones forman escenarios que deben ser considerados para garantizar un vuelo seguro y confiable. En términos generales, estas pueden clasificarse en tres categorías: fallas

de **actuador** con reducciones de hasta el 36 % del empuje en un motor o pérdida total de uno o varios rotores [3]; fallas de **sensor** lecturas erróneas del giroscopio, acelerómetro o GPS que pueden pasar desapercibidas durante varios ciclos de control [8, 9]; y **perturbaciones externas** de origen aerodinámico que modifican la dinámica del sistema de forma impredecible [10]. Sus consecuencias son documentadas: en diciembre de 2024, una formación de 500 drones perdió el control simultáneamente durante un espectáculo en Orlando, Florida, provocando la caída de unidades sobre el público y lesiones graves a un menor, según el reporte preliminar de la NTSB [11].

Esta situación genera un problema central: garantizar que un cuadricóptero mantenga estabilidad y continúe su misión ante la presencia simultánea de fallas en actuadores, sensores y perturbaciones externas, bajo restricciones computacionales propias de plataformas embebidas de bajo costo como el Crazyflie CF2X, donde el tiempo de respuesta y los recursos de cómputo disponibles son limitados.

Si bien los enfoques actuales resuelven parcialmente este problema los métodos basados en modelos detectan fallas mediante residuales [4, 6], las arquitecturas LSTM alcanzan una precisión del 79.48 % prediciendo fallas 30 segundos antes de un evento crítico [5], y las políticas de aprendizaje por refuerzo reportan tasas de éxito del 95 % en compensación de pérdidas de actuador [3], cada enfoque aborda de forma aislada uno solo de los tres tipos de falla y no considera simultáneamente la viabilidad computacional en hardware embebido. Esto limita su aplicación en escenarios reales donde las fallas pueden coexistir y los recursos de cómputo son restringidos. En consecuencia, existe una brecha metodológica no resuelta entre la capacidad de detección y compensación integrada de fallas y la eficiencia computacional necesaria para su despliegue en plataformas de bajo costo.

Formulación del problema

Limitada capacidad de los enfoques actuales para detectar y compensar de

manera integrada fallas en actuadores, sensores y perturbaciones dinámicas en UAV cuadricópteros, manteniendo una carga computacional viable para su implementación en plataformas embebidas de bajo costo.

Objetivos de investigación

- **Objetivo general:** Desarrollar un sistema integrado de detección y compensación de fallas en UAV cuadricóptero mediante de estados un estimador de estados y aprendizaje temporal ligero, validado en entorno simulado con escenarios de falla en actuadores, sensores y perturbaciones externas.

- **Objetivos específicos:**
 - Desarrollar, entrenar y comparar controladores basados en las arquitecturas Mamba SSM y LSTM mediante aprendizaje por imitación, evaluando su capacidad para el seguimiento de trayectorias en un UAV cuadricóptero.
 - Diseñar e implementar un mecanismo de compensación de fallas basado en aprendizaje por refuerzo profundo (PPO), que actúe sobre la salida del controlador base para mantener la estabilidad del sistema ante fallas de actuador.
 - Evaluar la tolerancia a fallas del sistema propuesto, utilizando como base el modelo de imitación con mejor desempeño, frente a pérdidas de potencia en el motor, considerando diferentes niveles de severidad.
 - Analizar el desempeño de los controladores en términos de precisión de seguimiento, capacidad de recuperación ante fallas y estabilidad en vuelo.

Justificación

El problema de la detección y compensación integrada de fallas en UAV cuadricóptero bajo restricciones computacionales resulta relevante por su impacto tanto en el ámbito metodológico como en el aplicativo.

Desde la perspectiva **metodológica**, este estudio contribuye al desarrollo de nuevas estrategias de control tolerante a fallas que combinan estimación de estados, aprendizaje temporal ligero y aprendizaje por refuerzo dentro de un mismo marco experimental. La evaluación de la arquitectura Mamba frente a LSTM en tareas de seguimiento de trayectoria y compensación de fallas plantea un reto técnico en términos de diseño de arquitecturas, eficiencia computacional y capacidad de generalización ante múltiples tipos de falla simultáneos. Resolver este desafío fortalece la literatura sobre modelos de espacio de estados aplicados al control de UAVs y abre la posibilidad de adaptar estas arquitecturas a otros sistemas robóticos con restricciones de cómputo similares [1, 12].

Desde la perspectiva **aplicativa**, disponer de un sistema de detección y compensación de fallas computacionalmente viable permite avanzar hacia el despliegue de UAVs cuadricóptero más seguros en entornos con presencia humana, como inspección, vigilancia y entrega de productos, donde la pérdida de control puede derivar en consecuencias graves [5, 11]. Asimismo, el conjunto de datos simulados generado en gym-pybullet-drones con escenarios de falla en actuadores, sensores y perturbaciones externas constituye un recurso reutilizable para investigaciones posteriores sobre FTC en plataformas de bajo costo.

Es por ello que esta investigación es relevante porque cierra la brecha entre el desempeño de las arquitecturas de aprendizaje profundo existentes y su viabilidad computacional para implementaciones embebidas, ofreciendo una alternativa metodológica

concreta y una base experimental reproducible para el diseño de sistemas de control tolerante a fallas en UAV cuadricóptero.

Alcance y limitaciones / restricciones

Con base en la justificación presentada, el **alcance** de esta investigación abarca el desarrollo y la validación de un sistema integrado de detección y compensación de fallas para un UAV tipo cuadricóptero en un entorno de simulación, empleando el simulador *gym-pybullet-drones* y el modelo Crazyflie CF2X como plataforma de referencia. No obstante, dado que la investigación se desarrolla en etapas, la presente entrega se limita a las fases de aprendizaje por imitación del controlador PID y compensación de fallas de actuador mediante PPO. Las fallas son incluidas directamente en el simulador, sin que el sistema deba identificarlas de forma autónoma. La incorporación de un estimador de estados para la detección y diagnóstico autónomo de fallas, así como la evaluación bajo fallas de sensor y perturbaciones externas, constituyen el alcance previsto para la **siguiente entrega**.

En cuanto a las **limitaciones** de la investigación, la validación del sistema se realiza exclusivamente en un entorno de simulación, por lo que la transferencia de los resultados a una plataforma física (*sim-to-real*) queda planteada como una línea de trabajo futuro [3, 10]. Asimismo, el modelo físico implementado en PyBullet constituye una aproximación de la dinámica de vuelo y no contempla fenómenos aerodinámicos de mayor complejidad, como turbulencias o la interacción entre rotores. Finalmente, la evaluación de la factibilidad de implementación en plataformas embebidas se limita al análisis de los requerimientos computacionales de los modelos desarrollados, sin considerar su despliegue y validación sobre hardware real.

CAPÍTULO I

REVISIÓN CRÍTICA DE LA LITERATURA

La investigación en Control Tolerante a Fallas (*Fault-Tolerant Control*, FTC) para vehículos aéreos no tripulados tipo cuadricóptero ha experimentado un importante desarrollo en los últimos años, impulsada por la necesidad de garantizar una operación segura y confiable frente a fallas en sensores, actuadores y perturbaciones externas [1, 2, 4]. Aunque los enfoques tradicionales basados en modelos dinámicos y técnicas de estimación de estados continúan siendo ampliamente utilizados, en los últimos años se ha observado un creciente interés por incorporar métodos de aprendizaje profundo y aprendizaje por refuerzo para mejorar la capacidad de detección, diagnóstico y compensación de fallas [1, 3, 5, 7]. En esta línea, diversos estudios han explorado estrategias que integran modelos físicos y técnicas de inteligencia artificial con el propósito de incrementar la robustez, adaptabilidad y confiabilidad de los UAV frente a condiciones de operación adversas.

1.1 Métodos Basados en Modelos y Observadores de Estado

Los métodos basados en modelos y observadores de estado constituyen una de las primeras estrategias desarrolladas para el Control Tolerante a Fallas (FTC) en vehículos aéreos no tripulados. Estos enfoques emplean modelos dinámicos del sistema junto con observadores o estimadores de estado para detectar, diagnosticar y, en algunos casos, compensar fallas mediante la comparación entre el comportamiento estimado por el modelo y la respuesta real del vehículo. Debido a su fundamento

teórico y a su bajo costo computacional, estas estrategias continúan siendo ampliamente utilizadas en sistemas embebidos; sin embargo, su desempeño depende en gran medida de la precisión del modelo dinámico disponible [1, 2].

Freddi et al. [4] propusieron un sistema integrado de Control Tolerante a Fallas para cuadricópteros basado en observadores dinámicos y un controlador pasivo por realimentación, con el objetivo de mantener la estabilidad del vehículo ante fallas en sensores y actuadores. El método emplea un banco de observadores de orden reducido para detectar y aislar fallas en sensores, mientras que las fallas en actuadores son identificadas mediante el análisis de señales residuales generadas por el modelo dinámico. Los experimentos realizados en MATLAB/Simulink demostraron que el controlador mantiene la estabilidad del cuadricóptero bajo distintos escenarios de falla, mostrando un mejor desempeño que un controlador convencional sin compensación. No obstante, la precisión del diagnóstico depende directamente del modelo linealizado utilizado por los observadores, lo que limita su desempeño frente a dinámicas altamente no lineales y perturbaciones aerodinámicas externas.

Desde una perspectiva enfocada únicamente en la estimación de estados Patan et al. [8] propusieron un esquema de estimación de estados basado en filtros de Kalman para incrementar la confiabilidad del sistema de navegación del cuadricóptero. Para ello compararon el filtro de Kalman extendido (EKF), el filtro de Kalman unscented (UKF) y el filtro de Kalman de cubatura (CKF), evaluando la precisión de cada uno durante el proceso de estimación. Los resultados mostraron que el CKF proporciona la menor incertidumbre y el menor error de estimación entre los métodos analizados. Sin embargo, la propuesta se limita a la detección de fallas en sensores y no incorpora mecanismos de compensación que permitan reconfigurar el controlador una vez identificada la falla.

Precisamente para integrar la estimación de fallas dentro del propio proceso de control, Du et al. [6] desarrollaron un esquema de FTC que combina el Control

de Rechazo Activo de Perturbaciones (ADRC, *Active Disturbance Rejection Control*) con un filtro de Kalman de dos etapas (TSKF, *Two-Stage Kalman Filter*). En este esquema, el TSKF estima el nivel de pérdida de efectividad de los actuadores, mientras que un Observador de Estado Extendido (ESO, *Extended State Observer*) compensa el error residual durante el control. Los resultados obtenidos evidenciaron una mejora en la estabilidad del sistema y en la capacidad de rechazo de perturbaciones respecto a un controlador (PID, *Proportional–Integral–Derivative*) convencional. No obstante, la validación solo se realizó mediante simulaciones en MATLAB/Simulink y el estudio considera exclusivamente fallas en actuadores, sin analizar fallas en sensores ni escenarios de perturbaciones aerodinámicas.

Aunque este tipo de estrategias mostró resultados favorables en simulación, su aplicación en plataformas físicas continuaba siendo un aspecto pendiente. En este contexto van Schijndel y Sun [13] propusieron un método de detección rápida de pérdida de efectividad en actuadores basado en un estimador de Kalman y un test de hipótesis. El algoritmo estima un factor estocástico de efectividad para cada actuador utilizando únicamente mediciones de revoluciones por minuto (RPM), giróscopos y acelerómetros obtenidas a bordo del UAV, permitiendo identificar el actuador defectuoso en tiempo real. La propuesta fue validada mediante vuelos experimentales y como parte de un sistema de Control Tolerante a Fallas activo, obteniendo tiempos de detección entre 30 y 130 ms sin registrar falsas alarmas ni detecciones perdidas. Sin embargo, el estudio se centra exclusivamente en la detección de fallas en actuadores y no incorpora mecanismos de diagnóstico para sensores ni estrategias de compensación frente a perturbaciones externas.

Además de la detección de fallas, otro desafío es mantener el desempeño del vehículo ante perturbaciones externas durante el vuelo. Bajo esta necesidad, surgieron estrategias que combinan el control por modo deslizante con observadores

no lineal. Ante esta necesidad, Chen et al. [14] propusieron un controlador adaptativo basado en modo deslizante integral, complementado con un observador de perturbaciones para compensar simultáneamente fallas en actuadores y perturbaciones externas. El observador estima las perturbaciones que afectan la dinámica del cuadricóptero, mientras que el controlador adapta la acción de control para mantener el seguimiento de la trayectoria deseada. Los resultados obtenidos mediante simulaciones demostraron una mejora en la precisión del seguimiento de trayectoria y en la estabilidad del sistema respecto a controladores convencionales. Sin embargo, la validación se realizó en un entorno de simulación y el estudio se centra en fallas en actuadores, sin considerar fallas en sensores ni escenarios de fallas múltiples.

Con el propósito de superar parcialmente la limitación asociada a la validación exclusivamente en simulación, Ahmadi et al. [15] propusieron un esquema de FTC activo basado en control por modo deslizante y un observador no lineal para estimar fallas de motor modeladas como incertidumbres paramétricas. El controlador fue validado tanto mediante simulaciones no lineales como en experimentos Hardware-in-the-Loop (HIL), donde fue comparado con los controladores *Incremental Nonlinear Dynamic Inversion* (INDI) y *Modified Incremental Nonlinear Dynamic Inversion* (MINDI), obteniendo un mejor desempeño en estabilidad y seguimiento de trayectoria. No obstante, el estudio continúa centrándose únicamente en fallas de actuadores y, aunque la validación HIL representa un avance respecto a la simulación convencional, el controlador no fue evaluado con un cuadricóptero en vuelo real.

A partir de los trabajos analizados, se observa que los métodos basados en modelos y observadores de estado constituyen el punto de partida para el desarrollo del Control Tolerante a Fallas en UAV, proporcionando mecanismos para la detección y compensación de fallas con un bajo costo computacional. Sin embargo, la efectividad de estas estrategias está estrechamente ligada a la disponibilidad

de modelos dinámicos precisos, condición que resulta difícil de satisfacer en escenarios reales donde existen incertidumbres, perturbaciones y dinámicas complejas. Además, la mayoría de las investigaciones se concentra en un único tipo de falla y valida sus propuestas únicamente mediante simulaciones, lo que limita su capacidad de generalización. Estas restricciones han impulsado el desarrollo de enfoques basados en aprendizaje profundo, capaces de aprender directamente a partir de los datos y reducir la dependencia del modelo físico del vehículo.

1.2 Detección de Fallas Basada en Aprendizaje Profundo

Debido a la dificultad de contar con modelos dinámicos precisos que representen todas las situaciones que puede enfrentar un UAV durante una misión, se ha incrementado el uso de técnicas de aprendizaje profundo para la detección de fallas. Estos métodos analizan directamente los datos obtenidos durante el vuelo para reconocer comportamientos inusuales, sin depender por completo de un modelo matemático del sistema.

Ozcan y Perkgoz [5] propusieron una arquitectura formada por un autoencoder LSTM para reducción de dimensionalidad con un clasificador BiLSTM para análisis bidireccional de las secuencias temporales, alcanzando una precisión del 79.48 % en la predicción de fallas hasta 30 segundos antes de que ocurra un descenso no controlado. No obstante, el dataset utilizado es reducido y sus datos de falla se encuentran en información compartida en línea por operadores, lo que limita la generalización del modelo. Además, al igual que los trabajos anteriores, la propuesta se restringe a la detección de fallas sin incorporar ningún mecanismo de compensación activo que permita al UAV mantener su estabilidad.

A diferencia de Ozcan y Perkgoz [5], quienes abordaron la predicción temprana de fallas a partir de múltiples señales de vuelo, Li et al. [16] propusieron un

modelo LSTM-OCSVM para detectar ataques ciberfísicos en señales relacionadas con la posición del UAV. El método utiliza una red LSTM para predecir el comportamiento esperado de las señales de los sensores y emplea las diferencias entre los valores predichos y reales como entrada de un clasificador OCSVM. Como este clasificador se entrena únicamente con datos de vuelo normal, no requiere ejemplos etiquetados de anomalías, lo que resulta útil en escenarios donde los datos de falla son escasos. La propuesta fue evaluada frente a cuatro tipos de ataques simulados: desviación de altura, alteración aleatoria de posición, denegación de servicio y repetición de datos. El modelo alcanzó una precisión promedio de 91.1 %, con un *recall* y una precisión de 90.7 %. Sin embargo, el estudio se limita a anomalías en los datos de posición y no considera fallas físicas en actuadores ni perturbaciones aerodinámicas. Además, la validación se realizó mediante ataques simulados, sin pruebas en condiciones reales de vuelo

En conjunto, los enfoques basados en aprendizaje profundo permiten identificar patrones temporales y comportamientos anómalos directamente a partir de los datos de vuelo, reduciendo la dependencia de modelos físicos detallados. Sin embargo, los trabajos revisados se concentran principalmente en detectar o anticipar la falla, sin abordar la respuesta que debe ejecutar el UAV después de identificarla. Esta limitación da lugar al uso de técnicas de aprendizaje por refuerzo, las cuales no solo permiten considerar el estado actual del vehículo, sino también aprender acciones de control orientadas a compensar los efectos de una falla durante la misión.

1.3 Control Tolerante a Fallas Basado en Aprendizaje por Refuerzo

Mientras que los enfoques de aprendizaje profundo revisados anteriormente se enfocan en detectar anomalías a partir de los datos de vuelo, el aprendizaje por refuerzo permite aprender acciones de control para responder ante una falla durante el vuelo del UAV. En este contexto, el aprendizaje por refuerzo (RL, *Reinforcement*

Learning) ha sido aplicado al Control Tolerante a Fallas en cuadricópteros debido a que permite entrenar políticas de control a partir de la interacción con el entorno, sin necesitar un modelo matemático.

Para abordar la recuperación del UAV ante alteraciones en las lecturas de los sensores, Fei et al. [17] propusieron *Learn-to-Recover*, un método que combina un controlador base robusto con una política de aprendizaje por refuerzo. Esta política funciona junto al controlador para recuperar el control del vehículo cuando recibe información incorrecta de los sensores durante ataques ciberfísicos, sin requerir un módulo independiente de detección y diagnóstico de fallas. El método fue evaluado frente a diferentes tipos de ataques en sensores y actuadores, como fallas totales, sesgos, alteraciones multiplicativas y reemplazo de señales. Los resultados mostraron una mejora importante respecto al controlador tradicional BlueBox. Por ejemplo, ante un sesgo severo de 32° en el sensor de actitud, el error promedio de posición pasó de 8382.9 mm con BlueBox a (-45.3) mm con la política de RL, con un error RMS de 48.2 mm frente a 10646 mm del controlador tradicional. De manera similar, ante el reemplazo de la señal del giroscopio por $0^\circ/s$, el error promedio se redujo de 123.3 mm a (-55.2) mm. Además, los resultados obtenidos en simulación fueron comprobados mediante pruebas en un UAV real, considerando ataques leves, como el reemplazo de la señal de un motor por un empuje fijo del 73% y la sustitución de la señal del giroscopio por $90^\circ/s$. Estas pruebas mostraron un comportamiento similar al observado en simulación. Sin embargo, aunque el estudio considera ataques tanto en sensores como en actuadores, su análisis se basa en alteraciones de sensores. Asimismo, no evalúa perturbaciones aerodinámicas, como ráfagas de viento, ni escenarios con fallas simultáneas, lo que limita su aplicación en condiciones de operación más complejas.

Buscando ampliar esta capacidad de compensación Liu et al.[7] propusieron

una estrategia para compensar fallas en actuadores mediante una política de aprendizaje por refuerzo basada en PPO (*Proximal Policy Optimization*) que complementa un controlador PID en cascada. La política genera acciones correctivas sin necesidad de identificar previamente el tipo o la magnitud de la falla. La propuesta fue evaluada en simulación y en pruebas reales con la plataforma Quanser QDrone, considerando fallas de sesgo y pérdidas de efectividad entre 10 % y 40 %. Los resultados mostraron que el error de seguimiento se mantuvo cercano al caso sin fallas; por ejemplo, ante una falla de magnitud 0.3, se obtuvo un error cuadrático medio (RMSE, *Root Mean Square Error*) de 0.059 m frente a 0.058 m en condiciones normales. Sin embargo, el estudio se limita a fallas moderadas en actuadores y no considera pérdida total de un rotor, fallas simultáneas, alteraciones en sensores ni perturbaciones externas.

Finalmente Hsu et al.[10] propusieron REFORMA, un enfoque de aprendizaje por refuerzo orientado a mejorar la respuesta del UAV frente a perturbaciones externas. Para ello, durante el entrenamiento incorporaron políticas adversarias que generan condiciones cada vez más exigentes, mientras un módulo de adaptación ajusta la intensidad de estas perturbaciones según el estado del vehículo.

El método fue evaluado en simulación en tareas de vuelo estacionario y cruce de compuerta. Los resultados mostraron que REFORMA obtuvo un mejor desempeño que otros enfoques de entrenamiento adversarial y que una política PPO convencional, especialmente en los escenarios con perturbaciones más exigentes. Sin embargo, la propuesta solo fue validada en simulación y las perturbaciones analizadas corresponden a alteraciones sobre las acciones de control, no a fallas físicas en sensores o actuadores. Además, su entrenamiento puede ser sensible al ajuste de sus parámetros.

En conjunto, los trabajos revisados muestran que el aprendizaje por refuerzo permite ir más allá de la detección de fallas, ya que puede aprender acciones de compensación para mantener la estabilidad y el seguimiento de trayectoria del UAV.

Sin embargo, cada propuesta se centra en un tipo particular de alteración fallas en sensores, pérdida de efectividad en actuadores, o perturbaciones externas sobre las acciones de control y la validación en hardware real, cuando existe, se realiza bajo condiciones acotadas y de baja severidad respecto a lo evaluado en simulación. Además, todavía es limitada la integración de fallas en sensores, actuadores y perturbaciones externas dentro de un mismo esquema de compensación.

1.4 Estrategias Híbridas de Adaptación en Tiempo Real

Una método emergente en el FTC para sistemas autónomos es el diseño de módulos de adaptación en tiempo real capaces de ajustar la política de control en tiempo real sin reentrenamiento, enfrentando dinámicas no vistas durante el entrenamiento.

Sohège et al.[18] propusieron un controlador supervisado basado en RL con controladores PID de bajo nivel (ganancia alta y baja) e incorporaron condiciones de estabilidad para evitar que la política ejecute acciones que vulneren restricciones de seguridad. Esta propuesta aporta seguridad durante el aprendizaje y la ejecución de la política. No obstante, no incluye un módulo de detección y diagnóstico que permita identificar el tipo o la magnitud de la falla, y su análisis se limita a fallas en actuadores.

Gran parte de los trabajos más recientes toma una idea que surgió originalmente en la robótica terrestre: el esquema de dos fases propuesto por Kumar et al.[19] para robots cuadrúpedos, conocido como RMA (*Rapid Motor Adaptation*). Allí, una política base se entrena en simulación con acceso completo a los parámetros del entorno; después, un segundo módulo aprende a estimar esos mismos parámetros usando solo el historial reciente de movimientos del robot, sin dicha información. Ambos módulos se ejecutan de forma independiente y a distinta velocidad, la política

principal a 100 Hz y el módulo de adaptación a 10 Hz, lo que permitió su implementación en un robot real (el A1 de Unitree) con recursos de cómputo limitados y sin necesidad de ajuste fino tras el entrenamiento.

Kim et al. [3] retoman directamente esta arquitectura y la trasladan al contexto aéreo, reemplazando el módulo original (una red convolucional) por un transformer, buscando una estimación más rápida ante cambios abruptos en el estado del vehículo. El salto de este paradigma de robots terrestres a cuadricópteros no es menor, ya que estos últimos son intrínsecamente más inestables y toleran mucho peor las oscilaciones que puede introducir un módulo de adaptación mientras aprende a reconocer la nueva situación. Su esquema conserva la misma lógica de dos fases: primero entrenan de forma conjunta una política de control y una red que codifica las condiciones del entorno en un vector reducido; luego, entrenan por separado el módulo de adaptación, que aprende a estimar ese mismo vector a partir de un breve historial de estado y acción del UAV, sin necesitar acceso a información privilegiada del sistema. Evaluado en simulación bajo una pérdida de efectividad del 30 % en un motor, el método alcanzó una tasa de éxito del 95 % y un error de posición (RMSE) de 0.129 m, superando tanto a una versión anterior de este enfoque basada en redes convolucionales (86 % de éxito, RMSE de 0.153 m) como a un controlador PID convencional (53 % de éxito). El trabajo también mostró que el controlador se adapta bien a configuraciones de UAV distintas a las usadas en el entrenamiento. Sus limitaciones son que no considera fallas en los sensores y que toda la validación se realizó en simulaciones, sin pruebas en escenarios reales.

Por otro lado, Giral, Gómez, Vinuesa y Le-Clainche [12] propusieron un Transformer que mapea directamente valores de referencia de bucle externo a comandos de control en UAVs de ala fija, eliminando los controladores internos y las capas de detección de fallas mediante destilación de conocimiento maestro-alumno. Zhang et al. [20], por su parte, presentaron un controlador de hovering único para

cuadricópteros con configuraciones físicas muy distintas (masa e inercia hasta 10 veces diferentes), validado tanto en simulación como en hardware real, que infiere parámetros físicos a partir de datos de vuelo recientes sin reentrenamiento. Aunque este trabajo es el más cercano al despliegue real dentro de esta sección, está orientado a la adaptación ante variaciones físicas del cuadricóptero y no a la tolerancia a fallas explícita en actuadores o sensores.

1.5 Consideraciones finales

A lo largo de esta revisión se analizaron trabajos relacionados con el Control Tolerante a Fallas en UAV, desde enfoques basados en aprendizaje por refuerzo hasta estrategias híbridas de adaptación en tiempo real. En conjunto, estos estudios reflejan una evolución desde políticas orientadas a compensar fallas específicas hacia arquitecturas con mayor capacidad de adaptación frente a condiciones no consideradas durante el entrenamiento.

Sin embargo, la mayoría de los trabajos aborda un solo tipo de alteración de forma aislada: fallas en actuadores [3, 7, 18], fallas en sensores [17], perturbaciones externas [10] o variaciones físicas en la dinámica del vehículo [12, 19, 20]. Por ello, aún son limitadas las propuestas que integran estos escenarios dentro de un mismo esquema de detección y compensación. Asimismo, aunque algunos estudios incluyen validaciones en UAV reales [7, 17, 20], otros se restringen a evaluaciones en simulación [3, 10, 12]. Además, las restricciones de cómputo propias de plataformas embebidas de bajo costo no suelen ser consideradas de manera explícita.

En consecuencia, se identifica la necesidad de desarrollar un enfoque integrado que combine estimación de estados y aprendizaje temporal ligero para detectar y compensar de forma conjunta fallas en sensores y actuadores, considerando también perturbaciones externas.

CAPÍTULO II

MARCO TEÓRICO

El presente capítulo presenta los fundamentos conceptuales y técnicos que sustentan esta investigación. En primer lugar, se describe la arquitectura y el modelo dinámico del UAV tipo cuadricóptero, con el fin de comprender las variables que intervienen en su comportamiento y control. En segundo lugar, se abordan las fallas que pueden afectar su estabilidad y desempeño durante el vuelo. Posteriormente, se presentan las arquitecturas de aprendizaje profundo utilizadas para el modelado secuencial: las redes LSTM (*Long Short-Term Memory*) y los modelos de espacio de estados selectivos Mamba (*Selective State Space Model, SSM*), que constituyen la base del controlador propuesto. Estas arquitecturas permiten aprender la relación entre el estado del UAV y las acciones de control, como alternativa a los controladores tradicionales, tales como el PID (*Proportional–Integral–Derivative*). Finalmente, se describe la arquitectura de compensación de fallas y el entorno de simulación empleado para la validación experimental preliminar.

2.1 Vehículos Aéreos No Tripulados tipo Cuadricóptero

Los vehículos aéreos no tripulados (UAV) de tipo cuadricóptero son plataformas aéreas que utilizan cuatro rotores distribuidos de forma simétrica alrededor de su estructura. Su uso se ha extendido en actividades como la inspección, vigilancia, entrega de objetos, búsqueda y rescate, debido a que pueden despegar y aterrizar verticalmente, mantenerse en vuelo estacionario y maniobrar en espacios reducidos [1]. Cada rotor actúa como un actuador del sistema, ya que recibe una señal de

velocidad angular y genera el empuje y el torque necesarios para controlar el movimiento del vehículo. Por esta razón, la estabilidad del cuadricóptero depende del funcionamiento coordinado de sus cuatro actuadores; una pérdida de rendimiento en cualquiera de ellos puede afectar la capacidad de control del UAV y poner en riesgo la misión [2].

2.1.1 Arquitectura y Principio de Funcionamiento

Un cuadricóptero está compuesto por cuatro motores ubicados en los extremos de una estructura en forma de cruz o de X. Dos de ellos giran en sentido horario y los otros dos en sentido antihorario. Esta disposición permite compensar los torques generados por la rotación de las hélices durante el vuelo estacionario.

El movimiento del vehículo se controla mediante la variación de la velocidad angular de sus cuatro rotores. A partir de esta combinación, el cuadricóptero puede regular el empuje vertical (*thrust*), el alabeo (*roll*), el cabeceo (*pitch*) y la guiñada (*yaw*). El empuje total aumenta cuando se incrementa de manera uniforme la velocidad de todos los motores. Por otro lado, las diferencias de velocidad entre rotores permiten generar los movimientos de alabeo y cabeceo, mientras que la guiñada se controla variando la velocidad entre los motores que giran en sentidos opuestos.

2.1.2 Modelo Dinámico

La dinámica de un cuadricóptero se describe mediante ecuaciones de movimiento en seis grados de libertad, divididas en traslación y rotación respecto a un marco de referencia inercial. La posición del vehículo se representa mediante las coordenadas (x, y, z) y su orientación mediante los ángulos de Euler: alabeo ϕ , cabeceo θ y guiñada ψ . Bajo la suposición de cuerpo rígido y masa m constante, las ecuaciones de traslación son:

$$m\ddot{x} = (\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) U_1 \quad (\text{II.1})$$

$$m\ddot{y} = (\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) U_1 \quad (\text{II.2})$$

$$m\ddot{z} = -mg + \cos \phi \cos \theta U_1 \quad (\text{II.3})$$

donde $U_1 = \sum_{i=1}^4 k_T \omega_i^2$ representa el empuje total generado por los rotores. Las variables involucradas en el modelo dinámico se resumen en la Tabla 2.1.

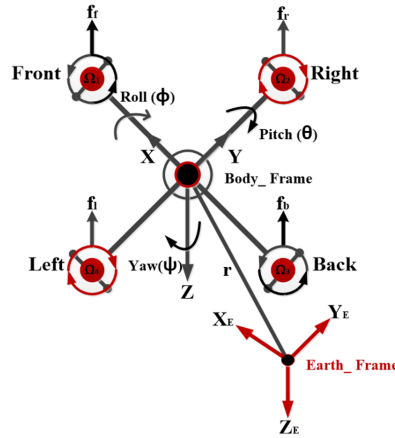


FIGURA 2.1: Estructura esquemática de un UAV tipo cuadricóptero y sus sistemas de coordenadas: ángulo de alabeo (*roll*, ϕ), cabeceo (*pitch*, θ) y guiñada (*yaw*, ψ) respecto al marco inercial. Tomado de [21].

Variable	Descripción
U_1	Empuje total generado por los rotores
k_T	Constante de empuje de cada rotor
ω_i	Velocidad angular del motor i
g	Aceleración gravitacional
I_{xx}, I_{yy}, I_{zz}	Momentos de inercia del UAV respecto a cada eje
U_2, U_3, U_4	Torques de control en los ejes de rotación

TABLA 2.1: Definición de variables en el modelo dinámico del cuadricóptero

A partir de las variables definidas en la Tabla 2.1 , las ecuaciones que describen la dinámica rotacional del cuadricóptero se expresan como:

$$I_{xx}\ddot{\phi} = \dot{\theta}\dot{\psi}(I_{yy} - I_{zz}) + U_2 \quad (\text{II.4})$$

$$I_{yy}\ddot{\theta} = \dot{\phi}\dot{\psi}(I_{zz} - I_{xx}) + U_3 \quad (\text{II.5})$$

$$I_{zz}\ddot{\psi} = \dot{\phi}\dot{\theta}(I_{xx} - I_{yy}) + U_4 \quad (\text{II.6})$$

Esta representación muestra que la dinámica del cuadricóptero es no lineal y que las variables que describen su movimiento están interrelacionadas, lo que representa un desafío para cualquier estrategia de control que busque mantener un desempeño adecuado ante perturbaciones externas o fallas en el sistema [6, 22].

2.2 Fallas en Sistemas UAV Cuadricóptero

Se denomina falla a cualquier desviación anormal en el comportamiento de un componente del sistema que reduzca su capacidad para cumplir la misión asignada. La literatura clasifica las fallas en los UAV cuadricóptero en tres categorías principales según el subsistema afectado [1, 2].

2.2.1 Clasificación de Fallas

Las **fallas en actuadores** corresponden a la pérdida parcial o total de efectividad en uno o más motores, originadas por desgaste mecánico, daño en las hélices, sobrecalentamiento o falla electrónica del controlador de velocidad. Matemáticamente, una falla de efectividad parcial en el motor i se modela mediante un factor $\eta_i \in [0, 1]$, donde $\eta_i = 1$ indica operación nominal y $\eta_i = 0$ representa pérdida total [14, 22]. Dado que el cuadricóptero tiene redundancia limitada, la degradación de un solo rotor afecta directamente los momentos de control disponibles y puede provocar pérdida de estabilidad si no se detecta y compensa a tiempo [3, 6].

Las **fallas en sensores** incluyen lecturas erróneas, sesgadas o ruidosas provenientes del giroscopio, el acelerómetro, el barómetro o el sistema de posicionamiento, originadas por interferencia electromagnética, vibración mecánica o ataques ciberfísicos [8, 9]. Mientras que las fallas en actuadores suelen reflejarse directamente en el comportamiento dinámico del cuadricóptero, las fallas en sensores pueden pasar desapercibidas durante varios ciclos de control, ya que el sistema continúa operando con información incorrecta hasta que la diferencia entre el estado estimado y el estado real se vuelve significativa.

Las **perturbaciones externas** de origen aerodinámico, como las ráfagas de viento, el efecto suelo o las variaciones de carga, pueden modificar la dinámica del sistema de manera impredecible. Aunque no corresponden a fallas internas del UAV, sus efectos pueden comprometer la estabilidad del vuelo de forma similar a una falla en actuadores. Por ello, los sistemas modernos de control tolerante a fallas suelen considerarlas dentro de sus procesos de diagnóstico y compensación [10, 14].

2.2.2 Escenarios de Fallas Simultáneas sobre la Dinámica del Vuelo

La presencia simultánea de múltiples tipos de falla representa un desafío para los sistemas de detección y compensación. Por ejemplo, una ráfaga de viento puede coincidir con la pérdida parcial de un motor o con una lectura degradada del acelerómetro, generando señales residuales ambiguas que dificultan el diagnóstico. Este tipo de escenario requiere estrategias capaces de distinguir el efecto de cada falla y responder de manera coordinada. Sin embargo, la mayoría de los enfoques desarrollados para un único tipo de falla presentan limitaciones cuando deben enfrentarse a este tipo de escenarios. [1, 2].

2.3 Arquitecturas de Aprendizaje Profundo para Control Secuencial

Las señales de estado del UAV, como la posición, velocidad, orientación y velocidad angular, forman secuencias temporales que permiten describir la evolución del vuelo. El aprendizaje profundo permite identificar patrones en estas secuencias a partir de datos históricos, sin depender directamente de un modelo dinámico explícito [5, 14]. En esta investigación se evalúan dos arquitecturas especializadas en el modelado de secuencias: la red LSTM, como línea base de comparación, y el modelo Mamba SSM, como propuesta principal.

2.3.1 Redes de Memoria de Largo y Corto Plazo (LSTM)

Las redes LSTM son una variante de las redes neuronales recurrentes diseñada para capturar dependencias temporales a largo plazo, superando el problema del desvanecimiento del gradiente que afecta a las RNN convencionales [5, 16]. La unidad LSTM incorpora tres compuertas de olvido, de entrada y de salida y un estado de

celda $\mathbf{c}_t$ que actúa como memoria a largo plazo. Las ecuaciones de actualización en el paso temporal t son:

- **Compuerta de olvido** (*forget gate*):

Determina qué fracción de la memoria a largo plazo $\mathbf{c}_{t-1}$ se conserva. Un valor cercano a 0 descarta la información pasada; un valor cercano a 1 la retiene completamente.

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{II.7})$$

- **Compuerta de entrada** (*input gate*):

Controla cuánta información nueva procedente de la entrada actual $\mathbf{x}_t$ y del estado anterior $\mathbf{h}_{t-1}$ se escribe en la memoria.

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{II.8})$$

- **Candidato al estado de celda:**

Genera los nuevos valores candidatos que podrían añadirse a la memoria. La función $\tanh$ acota los valores entre -1 y 1 , estabilizando el entrenamiento.

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (\text{II.9})$$

- **Actualización del estado de celda** (*cell state*):

El estado de celda se actualiza combinando la información relevante de la memoria previa, seleccionada por la compuerta de olvido, con la nueva información incorporada mediante la compuerta de entrada. Este mecanismo permite que la LSTM conserve información importante a lo largo de varios pasos temporales y modele dependencias de largo plazo.

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (\text{II.10})$$

Compuerta de salida (*output gate*):

Decide qué parte del estado de celda $\mathbf{c}_t$ se expone como salida hacia la capa siguiente o hacia el siguiente paso temporal.

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{II.11})$$

Estado oculto (*hidden state*):

Es la salida visible de la celda LSTM en el paso t . Actúa simultáneamente como salida hacia capas superiores y como entrada al siguiente paso temporal, codificando la memoria a corto plazo del modelo.

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (\text{II.12})$$

A partir de estas ecuaciones, se identifican las principales variables que intervienen en el modelo LSTM, las cuales se resumen en la Tabla 2.2.

Variable	Descripción
$\mathbf{x}_t$	Entrada en el paso de tiempo t
$\mathbf{h}_{t-1}$	Estado oculto en el paso anterior
σ	Función de activación sigmoide
$\odot$	Producto elemento a elemento (producto de Hadamard)

TABLA 2.2: Definición de variables en el modelo LSTM

Como la LSTM procesa la información de forma consecutiva en cada paso temporal, presenta menores posibilidades de paralelización durante el entrenamiento y puede generar una mayor latencia en la inferencia cuando se emplean ventanas temporales largas. En esta investigación, se utiliza como **línea base de comparación** una arquitectura LSTM compuesta por dos capas de 128 unidades ocultas, una ventana temporal de 50 pasos y un total de 216,388 parámetros entrenables.

2.3.2 Modelos de Espacio de Estados Selectivos: Mamba SSM

Mamba es una arquitectura de modelado secuencial basada en modelos de espacio de estados (SSM, *State Space Model*) diseñada para procesar secuencias de manera eficiente. Su característica principal es un mecanismo de selección que adapta sus parámetros en función de la información de entrada. A diferencia de los SSM clásicos cuyos parámetros son fijos durante toda la secuencia, Mamba puede priorizar o descartar información de acuerdo con su relevancia en cada instante temporal. El modelo continuo de espacio de estados que sirve como base se expresa mediante:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t), \quad \mathbf{y}(t) = \mathbf{C}\mathbf{h}(t) \quad (\text{II.13})$$

donde:

- $\mathbf{h}(t) \in \mathbb{R}^N$: estado oculto continuo.
- $\mathbf{x}(t)$: señal de entrada.
- $\mathbf{A}, \mathbf{B}, \mathbf{C}$: matrices de parámetros del sistema.

Para su aplicación en secuencias discretas, el sistema se discretiza mediante el método de retención de orden cero (ZOH, *Zero-Order Hold*) con escala temporal Δ :

$$\overline{\mathbf{A}} = e^{\Delta\mathbf{A}}, \quad \overline{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(e^{\Delta\mathbf{A}} - \mathbf{I}) \Delta\mathbf{B} \quad (\text{II.14})$$

resultando en la recurrencia discreta:

$$\mathbf{h}_t = \overline{\mathbf{A}} \mathbf{h}_{t-1} + \overline{\mathbf{B}} \mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C} \mathbf{h}_t \quad (\text{II.15})$$

En Mamba, los parámetros $\mathbf{B}$, $\mathbf{C}$ y Δ no son fijos sino que se definen como funciones de la entrada $\mathbf{x}_t$, lo que constituye el mecanismo de selección. Específicamente, en cada paso temporal se computan como proyecciones lineales de la entrada:

$$\mathbf{B}_t = W_B \mathbf{x}_t, \quad \mathbf{C}_t = W_C \mathbf{x}_t, \quad \Delta_t = \text{softplus}(W_\Delta \mathbf{x}_t + b_\Delta) \quad (\text{II.16})$$

El parámetro $\Delta_t > 0$ actúa como escala temporal de integración: valores grandes hacen que $\bar{\mathbf{A}}_t \rightarrow 0$, de modo que el modelo ignora el estado anterior y se concentra en la entrada actual; valores pequeños permiten propagar el estado oculto a través de pasos de tiempo distantes. Esta propiedad resulta relevante para el control de UAV, donde la importancia de señales como la velocidad angular, la posición o el error de seguimiento varía según la fase del vuelo.

Arquitectura de un bloque Mamba

Cada bloque Mamba encapsula el SSM selectivo dentro de una estructura que incluye: (i) dos proyecciones lineales de expansión que incrementan la dimensión interna por un factor *expand*; (ii) una convolución causal 1D de kernel reducido que introduce dependencia local sobre pasos de tiempo recientes sin recurrir a mecanismos de atención; (iii) el SSM selectivo que actualiza el estado oculto $\mathbf{h}_t$ según la Ec. (II.15); y (iv) una compuerta multiplicativa con activación SiLU que regula la información que fluye hacia la capa siguiente. Una conexión residual y normalización de capa envuelven todo el bloque para estabilizar el entrenamiento [23].

Modos de cómputo: entrenamiento e inferencia

Mamba opera en dos modos complementarios. Durante el entrenamiento, la recurrencia de la Ec. (II.15) se computa en paralelo sobre toda la secuencia mediante un *scan* asociativo, con complejidad $\mathcal{O}(L)$ en la longitud de la secuencia L . Durante la inferencia paso a paso, el modelo actualiza el estado oculto $\mathbf{h}_t$ de forma recurrente

con costo $\mathcal{O}(1)$ por paso, sin necesidad de materializar la secuencia completa. Este comportamiento lo diferencia de los Transformers, cuyo mecanismo de auto-atención tiene complejidad $\mathcal{O}(L^2)$, y lo equipara a la LSTM en costo de inferencia, con la ventaja adicional de que su estado oculto es de tamaño fijo independientemente de la longitud del contexto [23].

2.4 Aprendizaje por Imitación para Políticas de Control

El aprendizaje por imitación (*imitation learning*) es un paradigma en el que un agente aprende una política de control a partir de demostraciones proporcionadas por un experto, sin necesitar una función de recompensa ni un modelo dinámico del sistema [24]. En el control de UAV, el experto es un controlador clásico como un PID cuyo comportamiento se registra en forma de trayectorias estado-acción. A partir de estos datos, el modelo aprende a asociar cada observación con la acción de control correspondiente, con el propósito de reproducir el comportamiento del experto y operar como un controlador de vuelo completo.

En el marco del aprendizaje por imitación, se han desarrollado diversos métodos, entre los cuales se encuentran los enfoques directos, como el Behavior Cloning (BC) hasta variantes más avanzadas que incorporan interacción con el entorno o permiten inferir funciones de recompensa. A continuación, se describen las más representativas.

2.4.1 Behavioral Cloning

Se considera el enfoque más sencillo dentro del aprendizaje por imitación, ya que plantea el aprendizaje de la política como un problema de regresión supervisada a partir de un conjunto fijo de demostraciones generadas por el experto. Dado un

conjunto $\mathcal{D} = \{(\mathbf{s}_t, \mathbf{a}_t)\}_{t=1}^T$, donde $\mathbf{s}_t$ es el estado del sistema y $\mathbf{a}_t$ la acción correspondiente, el modelo f_θ se entrena minimizando el error cuadrático medio:

$$\mathcal{L}(\theta) = \frac{1}{T} \sum_{t=1}^T \|f_\theta(\mathbf{s}_{t-W:t}) - \mathbf{a}_t\|^2 \quad (\text{II.17})$$

donde $\mathbf{s}_{t-W:t}$ denota una ventana temporal de longitud W necesaria en modelos secuenciales y θ son los parámetros del modelo.

La principal ventaja de BC es su simplicidad, ya que no requiere interacción con el entorno ni la participación del experto durante el entrenamiento. Sin embargo, presenta como limitación la **deriva por covariables** (*covariate shift*), debido a que el modelo es entrenado con datos generados por el experto, pero en ejecución genera sus propias trayectorias, lo que puede provocar acumulación de errores a lo largo de la trayectoria [24].

2.4.2 DAgger

Con el fin de abordar el problema del *covariate shift*, DAgger (*Dataset Aggregation*) propone un proceso de entrenamiento iterativo que combina la ejecución de la política aprendida con la intervención del experto. En cada iteración, la política se ejecuta en el entorno para generar nuevas trayectorias. Posteriormente, el experto etiqueta las acciones correctas en los estados generados por la propia política durante su ejecución.

De esta manera, la política no solo aprende a imitar las trayectorias generadas por el experto, sino que también adquiere la capacidad de corregir sus propios errores al enfrentarse a estados generados de su propia interacción con el entorno. No obstante, este enfoque requiere la disponibilidad continua del experto durante el

proceso de entrenamiento, lo que puede implicar un costo elevado o limitar su aplicación en entornos reales, especialmente cuando el experto depende de la ejecución en sistemas físicos o de la participación continua de un operador humano.

2.4.3 Aprendizaje por Refuerzo Inverso

El aprendizaje por refuerzo inverso (IRL, *inverse reinforcement learning*) plantea un enfoque distinto al de la imitación directa. En lugar de reproducir las acciones del experto, busca estimar la función de recompensa $R(\mathbf{s}, \mathbf{a})$ que explica su comportamiento.

Una vez obtenida esta función, se utiliza aprendizaje por refuerzo para aprender una política de control. De esta forma, el modelo puede representar mejor el objetivo o la intención del experto y adaptarse a situaciones no observadas durante las demostraciones. Sin embargo, la estimación de la función de recompensa no es única, por lo que el problema resulta ambiguo y demanda un mayor costo computacional y requiere un mayor costo computacional.

2.4.4 GAIL

El método GAIL (*Generative Adversarial Imitation Learning*) integra el aprendizaje por imitación con un proceso de entrenamiento adversarial. En este enfoque, un discriminador aprende a distinguir entre trayectorias del experto y del agente, mientras que la política del agente se optimiza para engañar al discriminador .

Una ventaja importante es que no requiere conocer explícitamente las acciones del experto, sino únicamente sus trayectorias. Sin embargo, el entrenamiento puede ser inestable debido a la competencia entre la política y el discriminador.

2.4.5 Comparación de Métodos

Para resumir las diferencias entre los enfoques descritos, la Tabla 2.3 presenta una comparación de sus características principales.

Método	Interacción con entorno	Requiere experto online	Complejidad
BC	No	No	Baja
Dagger	Sí	Sí	Media
IRL	Sí	No	Alta
GAIL	Sí	No	Alta

TABLA 2.3: Comparación de métodos de aprendizaje por imitación

En aplicaciones de control de vuelo, BC destaca como una alternativa práctica para obtener una política inicial a partir de datos de un experto. No obstante, sus limitaciones en escenarios no vistos motivan la integración con otros enfoques, como el aprendizaje por refuerzo, para mejorar la robustez y capacidad de adaptación del sistema

2.5 Arquitectura de Control Tolerante a Fallas

El Control Tolerante a Fallas (FTC, *Fault-Tolerant Control*) tiene métodos que buscan mantener un funcionamiento adecuado del sistema ante la presencia de fallas. En los UAV de tipo cuadricóptero, el objetivo del FTC es preservar la estabilidad del vuelo y la capacidad de seguimiento de trayectoria frente a fallas en actuadores, sensores o perturbaciones externas [1].

Los metodos de FTC pueden clasificarse en dos categorías: **FTC pasivo** y **FTC activo**. Los sistemas de FTC pasivo incorporan robustez mediante el diseño previo del controlador sin necesitar detección ni reconfiguración en línea. Por otro lado, los sistemas de FTC activo utilizan un módulo de detección y diagnóstico de

fallas (FDD), para estimar la naturaleza de la falla y reconfigurar el controlador en tiempo real [4, 6, 15].

2.5.1 Arquitecturas de Control Jerárquico en FTC

En el contexto del Control Tolerante a Fallas, es común emplear arquitecturas de control jerárquicas o de múltiples niveles, donde cada componente cumple una función específica dentro del sistema. Generalmente, se distingue entre un **controlador nominal**, encargado de operar en condiciones normales, y un **módulo de compensación**, que actúa cuando se presentan fallas o perturbaciones.

El controlador nominal puede estar basado en modelos físicos o en técnicas de aprendizaje, y su función es generar la señal de control principal a partir del estado del sistema. Por otro lado, el módulo de compensación aplica acciones correctivas adicionales para reducir los efectos de las fallas, como la pérdida de efectividad en actuadores o perturbaciones externas.

De forma general, la señal de control en este tipo de arquitecturas puede expresarse como la suma de una acción nominal y una señal de compensación:

$$\mathbf{u}_t = \mathbf{u}_t^{\text{nom}} + \mathbf{u}_t^{\text{comp}} \quad (\text{II.18})$$

donde $\mathbf{u}_t^{\text{nom}}$ representa la acción generada por el controlador nominal y $\mathbf{u}_t^{\text{comp}}$ corresponde a la señal de corrección introducida por el módulo de compensación ante la presencia de fallas.

2.6 Entorno de Simulación: gym-pybullet-drones

gym-pybullet-drones es una plataforma de simulación de código abierto basada en el motor físico PyBullet, diseñada para modelar la dinámica de distintos UAV de tipo cuadricóptero mediante una interfaz compatible con OpenAI Gymnasium [25]. Este entorno permite representar de manera realista el comportamiento del vehículo, facilitando el desarrollo y evaluación de algoritmos de control en condiciones seguras y reproducibles.

La plataforma incorpora tres modelos de drones con diferentes características físicas y de propulsión: el Crazyflie 2.x en configuración en X (CF2X), el Crazyflie 2.x en configuración en cruz (CF2P) y un dron de carreras (RACE), incorporado en versiones posteriores del repositorio [25]. Sus principales parámetros se presentan en la Tabla 2.4, lo que permite comparar sus propiedades dinámicas y capacidades de vuelo. El control del sistema se ejecuta a una frecuencia de 48 Hz, mientras que la simulación física se actualiza internamente a 240 Hz. Esta diferencia de frecuencias permite capturar con mayor precisión los cambios rápidos en la dinámica del UAV, especialmente durante maniobras exigentes o ante la presencia de perturbaciones.

Parámetro	CF2X	CF2P	RACE
Configuración de brazos	X	+	X
Masa (kg)	0.030	0.030	0.830
Longitud de brazo (m)	0.0397	0.0397	0.109
Relación empuje/peso	2.25	2.25	4.17
Velocidad máxima (km/h)	30	30	200
Radio de hélice (m)	0.0231	0.0231	0.127
k_f (N·RPM ⁻²)	3.16×10^{-10}	3.16×10^{-10}	8.47×10^{-9}
k_m (N·m·RPM ⁻²)	7.94×10^{-12}	7.94×10^{-12}	2.13×10^{-11}
$I_{xx}=I_{yy}$ (kg·m ²)	1.40×10^{-5}	2.40×10^{-5}	3.11×10^{-3}
I_{zz} (kg·m ²)	2.17×10^{-5}	3.23×10^{-5}	3.11×10^{-3}

TABLA 2.4: Parámetros físicos y de propulsión de los modelos de drones disponibles en *gym-pybullet-drones* [25].

La *longitud de brazo* corresponde a la distancia entre el centro del cuerpo y cada motor, lo que influye directamente en el par de control disponible. El coeficiente de empuje k_f relaciona el empuje generado por una hélice con el cuadrado de su velocidad de rotación ($F = k_f \omega^2$), mientras que el coeficiente de par k_m vincula el torque reactivo con dicha velocidad ($\tau = k_m \omega^2$).

Por otro lado, la *relación empuje/peso* indica cuántas veces el empuje máximo puede superar al peso del vehículo, siendo un indicador clave de su capacidad de maniobra. Los momentos de inercia I_{xx} , I_{yy} e I_{zz} describen la resistencia del dron a rotar alrededor de sus ejes, afectando directamente su estabilidad y respuesta dinámica.

Finalmente, el modelo RACE presenta características significativamente distintas en comparación con los modelos Crazyflie, como una masa considerablemente mayor, hélices de mayor tamaño y una mayor relación empuje/peso, lo que le permite alcanzar velocidades más altas y realizar maniobras más agresivas.

La plataforma ofrece varias capacidades útiles para el desarrollo y evaluación de controladores aprendidos. En primer lugar, permite ejecutar controladores clásicos, como PID (*Proportional–Integral–Derivative*), y registrar los pares estado-acción que generan, facilitando la creación de conjuntos de datos para aprendizaje por imitación. Asimismo, permite desplegar políticas aprendidas en **lazo cerrado**, es decir, donde las acciones generadas por el controlador se aplican directamente al sistema y las nuevas mediciones del estado se retroalimentan continuamente como entrada del modelo, sustituyendo al controlador clásico y permitiendo evaluar métricas relacionadas con el seguimiento de trayectoria. Finalmente, permite simular fallas en los actuadores al modificar la efectividad de los motores durante la ejecución, lo que hace posible analizar la respuesta de los controladores ante pérdidas parciales o totales de empuje [25].

CAPÍTULO III

MARCO METODOLÓGICO

La presente investigación plantea un *pipeline* metodológico para comparar las arquitecturas LSTM y Mamba SSM como controladores de vuelo aprendidos mediante imitación, y analizar su desempeño junto con un compensador basado en PPO ante fallas de actuador en un UAV cuadricóptero. La propuesta busca reemplazar un controlador clásico, como el PID, por políticas aprendidas que puedan operar sin utilizar de forma explícita el modelo dinámico del vehículo. Asimismo, incorpora un nivel adicional de compensación mediante aprendizaje por refuerzo para mejorar la respuesta del sistema cuando se presentan fallas.

El desarrollo se realiza en el simulador *gym-pybullet-drones*, utilizando el modelo CF2X. Este entorno permite generar datos de entrenamiento controlados, ejecutar las políticas en lazo cerrado e introducir fallas de manera reproducible, sin requerir hardware físico. En esta primera etapa, las fallas de actuador se emplean como caso de validación inicial de la metodología.

El capítulo describe el procedimiento seguido para desarrollar y evaluar la propuesta. El flujo se organiza en tres fases: (i) generación del conjunto de datos de imitación a partir de un controlador PID de referencia; (ii) entrenamiento de los controladores LSTM y Mamba SSM, junto con el compensador PPO; y (iii) evaluación en lazo cerrado sobre trayectorias reproducibles, en condiciones nominales y bajo fallas de actuador. Esta organización permite que las diferencias observadas se relacionen principalmente con la arquitectura evaluada y no con cambios en el procedimiento experimental.

Dado que la investigación aún se encuentra en desarrollo y los experimentos finales no han concluido, este capítulo presenta la metodología general y las etapas previstas para su implementación. En esta primera entrega, la evaluación se centra en verificar el funcionamiento integrado de los controladores aprendidos y del compensador PPO ante fallas de actuador previamente inyectadas en el simulador. La incorporación de un estimador de estados encargado de identificar la presencia de fallas durante la operación, se abordará en una etapa posterior. Asimismo, se extenderá la evaluación a otros tipos de fallas, perturbaciones externas y escenarios en los que varias fallas ocurran de manera simultánea.

3.1 Descripción General de la Propuesta

La investigación adopta un enfoque cuantitativo y experimental. Es cuantitativa porque el desempeño de los controladores se mide mediante indicadores numéricos, como el error de posición, la tasa de éxito, el número de parámetros y la latencia de inferencia. Estas métricas permiten analizar las diferencias entre las arquitecturas de manera objetiva. Por otro lado, tiene carácter experimental porque las arquitecturas LSTM y Mamba SSM se entrenan y evalúan bajo las mismas condiciones controladas de simulación, con el fin de observar cómo influye la arquitectura del controlador en su desempeño.

El estudio presenta un diseño comparativo mediante simulación computacional. Ambas arquitecturas aprenden a imitar el comportamiento de un controlador PID y, posteriormente, se evalúan en lazo cerrado, tanto de forma individual como en combinación con el compensador PPO ante escenarios de falla.

Antes de presentar la arquitectura de control propuesta, se establecen las condiciones bajo las cuales se realizará la comparación entre los controladores LSTM y Mamba SSM.

■ **Variable Independiente:**

Es la arquitectura del controlador aprendido por imitación, que toma dos valores: red LSTM de dos capas con 128 unidades ocultas, y modelo Mamba SSM con dos capas de $d_{\text{model}} = 128$. Ambas arquitecturas se entrenan con el mismo dataset, los mismos hiperparámetros y la misma ventana temporal de 50 pasos, de modo que la única diferencia entre las condiciones experimentales sea el enfoque de modelado secuencial.

■ **Variables Dependientes:**

Las variables se organizan en dos niveles según la etapa del sistema evaluada.

• **Nivel 1: Imitación del controlador PID (BC):**

Métrica	Descripción
Error de validación (MSE)	Error cuadrático medio sobre el conjunto de prueba; mide la fidelidad de imitación del controlador PID en datos no vistos.
Número de parámetros entrenables	Medida del costo computacional y capacidad del modelo (LSTM vs. Mamba SSM).
Latencia de inferencia	Tiempo de cómputo requerido por cada paso de control a 48 Hz.
Tasa de éxito en vuelo nominal	Proporción de trayectorias completadas sin falla dentro del umbral de aceptación.
Error de posición final (nominal)	Distancia euclidiana al punto destino al finalizar el episodio sin falla activa.

TABLA 3.1: Métricas de Nivel 1 - Evaluación de la Política de Imitación

- **Nivel 2: Compensación de falla de motor (PPO residual):**

Métrica	Descripción
Tasa de recuperación	Proporción de episodios en que el dron completa la trayectoria o aterriza de forma controlada bajo pérdida de potencia activa $\left(\frac{\text{llegó} + \text{aterrizó}}{N}\right)$.
Tasa de crash	Proporción de episodios con caída no controlada bajo falla activa $\left(\frac{\text{cayó}}{N}\right)$.
Porcentaje de misión completado	Fracción promedio de waypoints alcanzados antes de que el episodio termine, independientemente del resultado.
Error de posición final (falla)	Distancia euclidiana al punto destino al finalizar el episodio con falla activa.

TABLA 3.2: Métricas de Nivel 2 - Evaluación de la compensación ante fallas de motor

- **Variables de Control:**

Se mantienen constantes en todos los experimentos para garantizar una comparación justa entre las arquitecturas evaluadas.

Variable de Control	Valor / Condición
Entorno de simulación	<i>gym-pybullet-drones</i> , versión fija con motor de física PyBullet.
Modelo de dron	Crazyflie 2.X (CF2X), parámetros aerodinámicos y URDF fijos.
Conjunto de datos de entrenamiento	800 episodios generados con <code>DSLPIDControl</code> ; 18 señales de estado y 4 salidas de motor a 48 Hz.
Hiperparámetros de optimización	Optimizador Adam, misma tasa de aprendizaje, tamaño de lote y número de épocas para ambas arquitecturas.
Trayectorias de evaluación	5 trayectorias con coordenadas fijas definidas en el módulo de comparación.
Escenario de falla	Falla en el motor 0; factor de efectividad evaluado en $\eta_0 \in \{0.85, 0.90, 0.95\}$.

TABLA 3.3: Variables de control que se mantienen constantes en todos los experimentos

3.2 Pipeline Metodológico Propuesto

En esta sección se presenta el pipeline metodológico propuesto, el cual resume las etapas para el desarrollo y evaluación del sistema. Posteriormente, cada una de estas etapas será descrita en detalle.

1. Generación del dataset de imitación

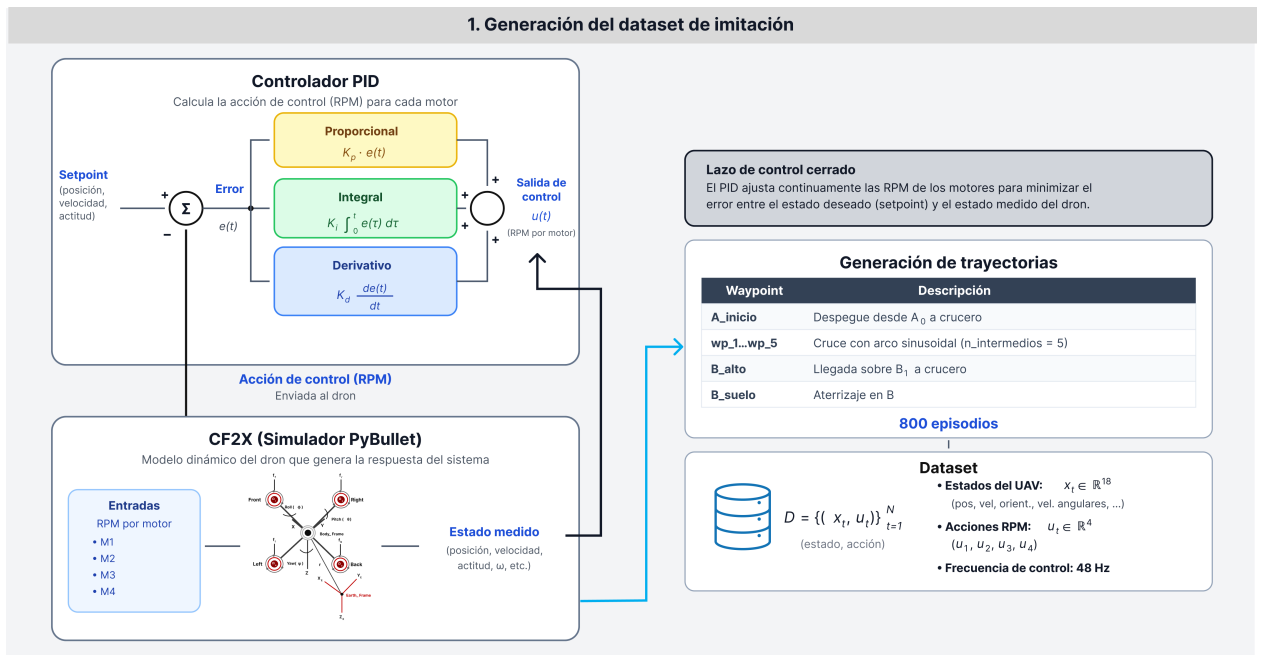


FIGURA 3.1: Parte 1 - Generación del dataset de imitación.

2. Entrenamiento por Behavioral Cloning

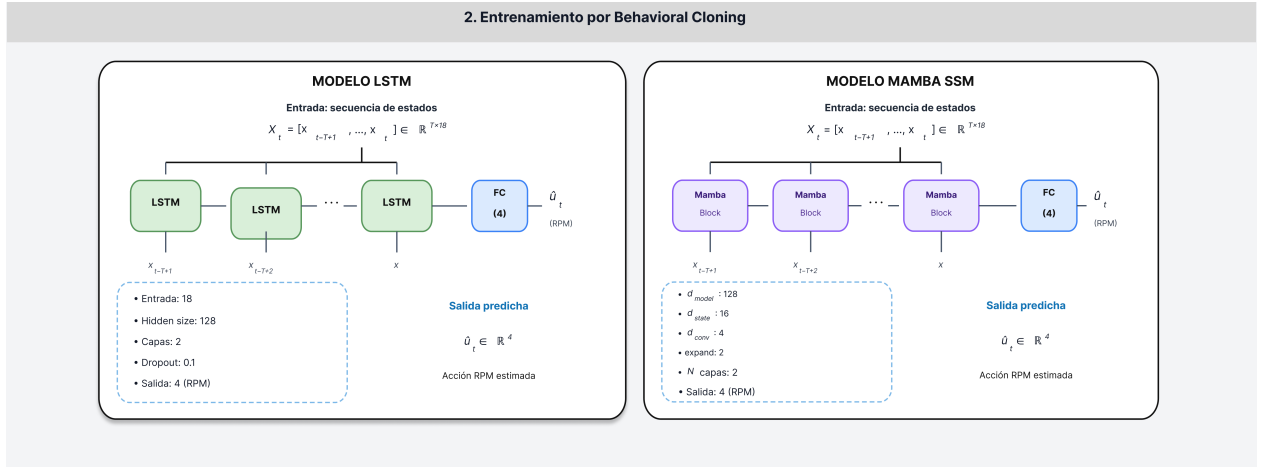


FIGURA 3.2: Parte 2 - Entrenamiento por Behavioral Cloning.

3. Entrenamiento del compensador PPO

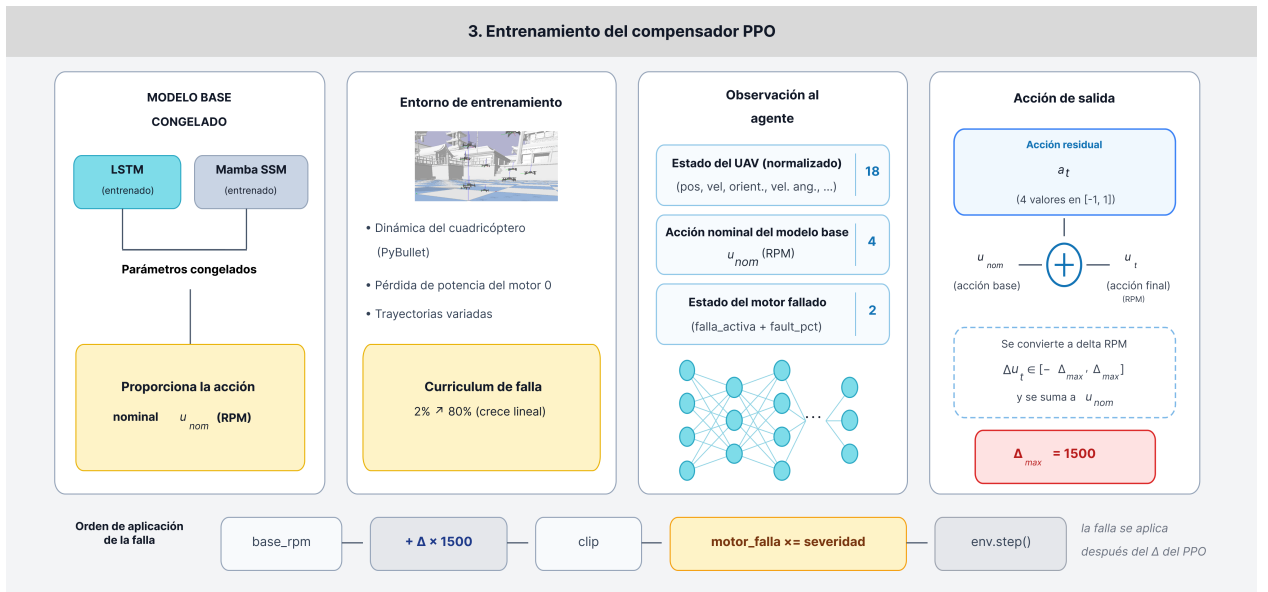


FIGURA 3.3: Parte 3 - Entrenamiento del compensador PPO.

4. Evaluación

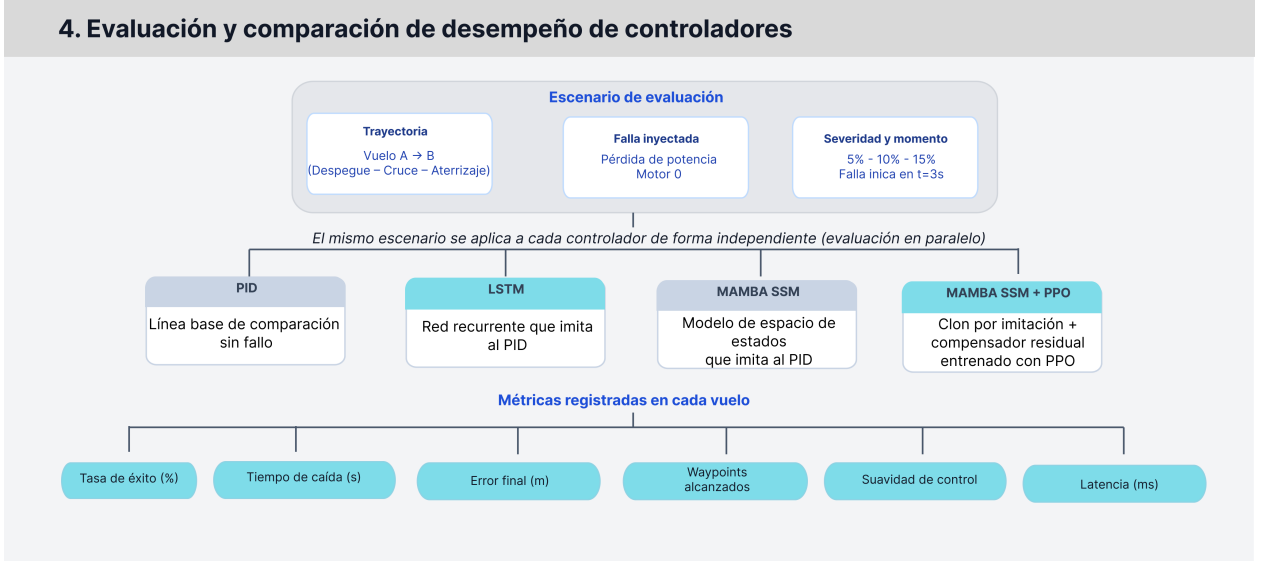


FIGURA 3.4: Parte 4 - Evaluación y comparación de desempeño de controladores

A continuación, se describe el pipeline metodológico presentado en las Figuras 3.1-3.4. Se organiza en etapas secuenciales y se detalla con el objetivo de facilitar su futura implementación completa.

3.2.1 Generación del Dataset de Imitación

Se emplea el controlador PID de referencia `DSLPIDControl`, incluido en la librería *gym-pybullet-drones* y desarrollado por el Dynamic Systems Lab de la Universidad de Toronto (DSL-UTIAS) [26]. Dicho controlador se ejecuta en el entorno *gym-pybullet-drones* para generar el conjunto de datos de entrenamiento, a lo largo de 800 episodios de vuelo aleatorio. En cada episodio, el dron parte de una posición inicial A_{suelo} seleccionada aleatoriamente dentro del espacio de vuelo $[-2.0, 2.0]$ m en los ejes x e y , con una distancia mínima de 0.8 m respecto al destino B_{suelo} (wp_8).

La trayectoria se define mediante **8 waypoints secuenciales**, los cuales se describen en la Tabla 3.4 y se representan en la Figura 3.5.

Waypoint	Descripción
wp ₁	Punto <i>A</i> elevado a la altitud de crucero ($z = 1.2$ m).
wp ₂ -wp ₆	Cinco puntos intermedios generados mediante interpolación lineal con variación sinusoidal en el eje vertical ($z = 1.2 + 0.1 \sin(t\pi)$ m).
wp ₇	Punto <i>B</i> a la misma altitud de crucero.
wp ₈	Punto <i>B</i> al nivel de aterrizaje ($z = 0.1$ m).

TABLA 3.4: Definición de los *waypoints* de la trayectoria.

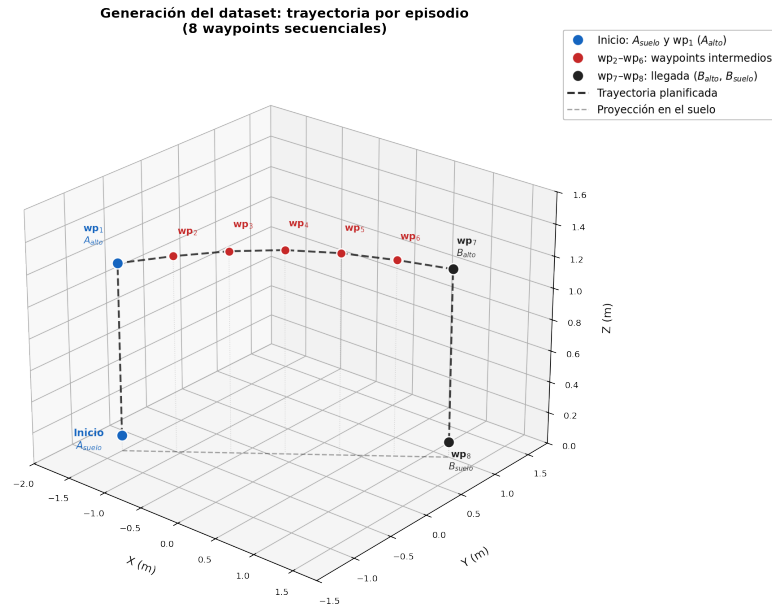


FIGURA 3.5: Waypoints secuenciales por episodio.

Durante la ejecución de cada episodio, el dron sigue la trayectoria definida por los *waypoints* mediante la acción continua del controlador.

En cada paso de control, el controlador PID utiliza el estado actual del dron para calcular las RPM de los cuatro motores y dirigirlo hacia el *waypoint* activo. Tanto el estado como las acciones generadas se registran, formando los datos de entrada y salida del dataset de imitación.

Las variables que conforman el estado del UAV se detallan en la Tabla 3.5.

Variable	Descripción
(x, y, z)	Posición absoluta del dron en el espacio.
(v_x, v_y, v_z)	Velocidad lineal en los ejes x , y y z .
(ϕ, θ, ψ)	Orientación del dron en ángulos de Euler (roll, pitch y yaw).
$(\omega_x, \omega_y, \omega_z)$	Velocidad angular en los ejes x , y y z .
(x_{wp}, y_{wp}, z_{wp})	Posición del <i>waypoint activo</i> , actualizada cuando el dron alcanza el punto dentro de un radio de aceptación de 0.10 m.
(e_x, e_y, e_z)	Error de posición respecto al <i>waypoint activo</i> .

TABLA 3.5: Variables que conforman el vector de estado del UAV

Al finalizar los 800 episodios, se obtiene el dataset de imitación a partir de las muestras **estado-acción** registradas en cada paso de control. La Tabla 3.6 resume sus principales características. Este conjunto se guarda en formato CSV para su posterior uso en el entrenamiento de los modelos de imitación propuestos.

Parámetro	Valor
Número de episodios totales	800
Episodios de entrenamiento	640
Episodios de validación	160
Frecuencia de control	48 Hz
Duración máxima por episodio	20 s (960 pasos)
Dimensión del vector de estado	18
Dimensión del vector de acción	4 (RPM motores)
Rango de RPM	[9 440, 21 700] RPM
Espacio de vuelo (x, y)	$[-2.0, 2.0]$ m
Altitud de crucero	1.2 m

TABLA 3.6: Características del dataset de imitación generado.

3.2.2 Preprocesamiento y Normalización

Posteriormente, una vez generado el dataset de 800 episodios, los datos se preprocesan en dos etapas antes de ser utilizados para el entrenamiento de los 2 modelos propuestos (LSTM y MAMBA SSM)

1. Partición del dataset:

Los 800 episodios se dividen en un conjunto de entrenamiento (80 %, 640 episodios) y un conjunto de validación (20 %, 160 episodios). Esta partición se realiza a nivel de episodio y no de filas individuales utilizando una semilla fija **42**, para garantizar la reproducibilidad de los resultados, asegura que ningún episodio aparezca simultáneamente y evita la filtración de información temporal entre entrenamiento y validación.

2. Normalización Z-score:

Posteriormente, se aplica una normalización tipo Z-score a las señales del dataset. Para ello, los parámetros de normalización **media** μ_j y **desviación estándar** σ_j se calculan a partir del conjunto de entrenamiento para cada una de las 22 señales (18 de entrada y 4 de salida). Estos parámetros se aplican tanto al conjunto de entrenamiento como al de validación mediante la transformación Z-score definida en la Ecuación III.1.

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j} \quad (\text{III.1})$$

Este procedimiento asegura que todas las variables tengan media cero y varianza unitaria, lo que favorece la estabilidad y la velocidad de convergencia del entrenamiento.

3.2.3 Entrenamiento por imitación mediante clonación de comportamiento (Behavioral Cloning)

Una vez completado el preprocesamiento del dataset, se procede al entrenamiento de los modelos mediante aprendizaje por imitación. En este contexto, se proponen y comparan dos arquitecturas de aprendizaje profundo para la tarea de imitación: una red neuronal recurrente basada en **LSTM** y un modelo de espacio de estados basado en **Mamba SSM**.

Ambos modelos reciben como entrada secuencias temporales del estado del UAV construidas mediante ventanas deslizantes de 50 pasos consecutivos y producen como salida las RPM normalizadas de los cuatro motores.

Antes de describir el proceso de entrenamiento, es necesario definir dos elementos fundamentales: la forma en que se construyen las muestras a partir del dataset y la función objetivo que guía el aprendizaje.

1. Construcción de las muestras

Cada muestra se define como un par (X_t, y_t) , donde:

- X_t : corresponde a la secuencia de entrada
- y_t : es la acción de salida generada por el controlador PID en el mismo instante

Entrada: La secuencia de entrada en el paso de control t agrupa los $T = 50$ estados consecutivos más recientes:

$$X_t = [x_{t-T+1}, \dots, x_t] \in \mathbb{R}^{50 \times 18} \quad (\text{III.2})$$

donde $t = 0, 1, \dots, n_e$ indexa cada paso de control del episodio a una frecuencia de $f_c = 48Hz$ (un paso cada $\Delta t = 0.021s$), y cada $x_t \in \mathbb{R}^{18}$ contiene las señales

de posición, velocidad, actitud, velocidad angular, posición objetivo y error de posición registradas en ese instante.

Salida: La etiqueta asociada es $y_t \in \mathbb{R}^4$, correspondiente a las RPM normalizadas de los cuatro motores generadas por el controlador PID en el instante t .

Generación de muestras:

Las muestras se generan a partir de una ventana deslizante de paso unitario. Es decir, para cada instante t del episodio se genera un par distinto (X_t, y_t) , lo que produce un total de:

$$N = \sum_{e=1}^{640} n_e \quad (\text{III.3})$$

donde n_e es la duración en pasos del episodio e .

Condición inicial:

Cuando $t < T$ (inicio del episodio), la ventana se completa con ceros al inicio (*zero-padding*), garantizando que todas las muestras mantengan la misma dimensión de entrada $\mathbb{R}^{50 \times 18}$.

2. Función de pérdida

El entrenamiento se realiza minimizando el error cuadrático medio (MSE) entre las RPM predichas $\hat{y}$ y las generadas por el controlador PID y :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|^2 \quad (\text{III.4})$$

3.2.3.1. Proceso de entrenamiento

El entrenamiento se lleva a cabo durante 150 épocas. En cada época, el modelo procesa lotes (*batches*) de 128 muestras, realizando una propagación hacia adelante

para predecir las RPM, seguida del cálculo de la función de pérdida y la propagación hacia atrás para actualizar los parámetros del modelo.

Para prevenir el problema de **exploding gradient**, típica de modelos secuenciales con ventanas de 50 pasos, se aplica recorte de gradientes (*gradient clipping*) con un umbral de 1.0. La optimización se realiza mediante el algoritmo Adam con una regularización L2 de $\lambda = 1 \times 10^{-5}$ para evitar sobreajuste, mientras que la tasa de aprendizaje se ajusta dinámicamente mediante el schedule **ReduceLROnPlateau**, el cual reduce el *learning rate* cuando el error de validación no mejora durante 10 épocas consecutivas.

El modelo con menor error de validación se guarda y se emplea en la etapa de evaluación en bucle cerrado dentro del simulador PyBullet. Por su parte, los hiperparámetros de entrenamiento, comunes a ambas arquitecturas, se resumen en la Tabla 3.7.

Parámetro	Valor
Ventana temporal (T)	50 pasos
Tamaño de batch	128
Épocas	150
Learning rate inicial	1×10^{-3}
Optimizador	Adam ($\text{weight_decay} = 1 \times 10^{-5}$)
Scheduler	ReduceLROnPlateau ($\text{patience} = 10$, $\text{factor} = 0.5$)
Gradient clipping	1.0
Dropout	0.1
Función de pérdida	MSE (Ec. III.4)

TABLA 3.7: Hiperparámetros de entrenamiento comunes a LSTM y Mamba SSM.

A continuación, se describen las arquitecturas utilizadas para modelar dicha función de control.

3.2.3.2. Modelo LSTM

La red neuronal recurrente de tipo LSTM se diseña para procesar la secuencia de entrada $X_t \in \mathbb{R}^{50 \times 18}$ y producir las RPM normalizadas de los cuatro motores $\hat{u}_t \in \mathbb{R}^4$.

La arquitectura se compone de dos etapas:

1. Extracción de características temporales:

La secuencia X_t se procesa mediante dos capas LSTM consecutivas, cada una con un estado oculto de dimensión $h = 128$. Se aplica *dropout* de 0.1 entre capas para reducir el sobreajuste. De la salida de la segunda capa se extrae la representación del último paso de tiempo $h_T \in \mathbb{R}^{128}$, que captura la información temporal acumulada de los 50 pasos de la ventana.

2. Proyección a la acción de control:

El vector h_T se pasa por una **capa completamente conectada** con una capa intermedia de 64 unidades, activación ReLU y *dropout* de 0.1, que produce la predicción final:

$$\hat{u}_t = W_2 \text{ReLU}(W_1 h_T + b_1) + b_2 \in \mathbb{R}^4 \quad (\text{III.5})$$

El modelo cuenta con un total de **216 388 parámetros entrenables**.

3.2.3.3. Modelo Mamba SSM

El modelo Mamba adapta los modelos de espacio de estados selectivos para la **imitación del controlador**.

La arquitectura se compone de tres etapas:

1. Proyección de entrada:

Dado que los bloques Mamba requieren una dimensión fija d_{model} , la secuencia de entrada $X_t \in \mathbb{R}^{50 \times 18}$ se proyecta primero a la dimensión del modelo mediante una capa lineal:

$$Z^{(0)} = X_t W_{\text{proj}} \in \mathbb{R}^{50 \times 128} \quad (\text{III.6})$$

2. Procesamiento con bloques Mamba:

La secuencia proyectada se procesa mediante $L = 2$ bloques Mamba consecutivos. Cada bloque implementa un mecanismo de selección de estado, seguido de una conexión residual y normalización de capa:

$$Z^{(\ell)} = \text{LayerNorm}(Z^{(\ell-1)} + \text{Mamba}(Z^{(\ell-1)})), \quad \ell = 1, 2 \quad (\text{III.7})$$

Cada bloque emplea los siguientes parámetros:

Parámetro	Valor
d_{model}	128
d_{state}	16
d_{conv}	4
expand	2

TABLA 3.8: Parámetros de cada bloque Mamba.

3. Proyección a la acción de control:

Al igual que en la LSTM, la representación del último paso de tiempo $Z_T^{(2)} \in \mathbb{R}^{128}$ se proyecta a través de una capa completamente conectada ($128 \rightarrow 64 \rightarrow 4$) para obtener la predicción de las RPM normalizadas de los 4 motores en el instante t ($\hat{u}_t \in \mathbb{R}^4$). El modelo cuenta con **244 420 parámetros entrenables**.

3.2.4 Entrenamiento del Compensador PPO

El compensador de fallas basado en PPO se entrena de forma independiente al controlador por imitación, a través de un segundo proceso de optimización que utiliza el simulador *gym-pybullet-drones* con inyección de fallas de actuador.

En esta etapa, el controlador Mamba previamente entrenado se mantiene con parámetros congelados y proporciona la acción nominal, mientras que el agente PPO aprende exclusivamente la política de compensación $g_{\text{PPO}}(\mathbf{s}_t, \hat{\boldsymbol{\eta}})$.

3.2.4.1. Entorno de entrenamiento

El entorno de entrenamiento implementa la interfaz `gymnasium.Env` y expone al agente PPO un espacio de observación de 24 dimensiones:

$$o_t = \left[\underbrace{x_t^{\text{norm}}}_{18}, \underbrace{\hat{u}_t^{\text{Mamba}}}_{4}, \underbrace{f_{\text{activa}}, f_{\text{pct}}}_{2} \right] \in \mathbb{R}^{24} \quad (\text{III.8})$$

donde $x_t^{\text{norm}} \in \mathbb{R}^{18}$ es el estado normalizado del dron, $\hat{u}_t^{\text{Mamba}} \in \mathbb{R}^4$ es la predicción base del modelo Mamba en ese instante, $f_{\text{activa}} \in \{0, 1\}$ indica si la falla está activa, y $f_{\text{pct}} \in [0, 1]$ es el porcentaje de potencia perdida.

El espacio de acción es continuo:

$$\delta_t \in [-1, 1]^4 \quad (\text{III.9})$$

que se escala a una corrección de RPM mediante $\Delta_{\text{máx}} = 1500$:

$$\text{RPM}_{\text{final}} = \text{clip}(\hat{u}_t^{\text{RPM}} + \delta_t \cdot 1500, \text{RPM}_{\text{mín}}, \text{RPM}_{\text{máx}}) \quad (\text{III.10})$$

Antes de la falla ($t < t_{\text{falla}}$), el agente PPO no interviene y el dron es controlado exclusivamente por Mamba. Desde $t = t_{\text{falla}}$, PPO aplica la corrección δ_t sobre la acción base de Mamba y la falla se inyecta degradando el motor 0 con la severidad del episodio.

3.2.4.2. Incremento progresivo de falla

La magnitud máxima de la falla aumenta linealmente desde el 2 % hasta el 80 % de pérdida de potencia durante el primer 75 % de los pasos de entrenamiento, manteniéndose en el 80 % durante el 25 % restante:

$$f_{\text{máx}}(k) = 0.02 + \min\left(\frac{k}{0.75 K}, 1\right) \times (0.80 - 0.02) \quad (\text{III.11})$$

donde k es el paso de entrenamiento actual y $K = 1\,000\,000$ es el total de pasos. En cada episodio, la severidad se aleatoriza uniformemente en $[0.02, f_{\text{máx}}(k)]$, exponiendo progresivamente al agente a escenarios más difíciles conforme avanza el entrenamiento.

3.2.4.3. Función de recompensa

La función de recompensa se aplica tras la falla ($t \geq t_{\text{falla}}$) y está compuesta por los siguientes términos:

- **Costo temporal:** $r_{\text{tiempo}} = -0.01$ por paso, para evitar que el agente permanezca inactivo.
- **Estabilidad:** penalización proporcional a los ángulos roll, pitch y a la velocidad angular:

$$r_{\text{estab}} = -2(|\phi| + |\theta|) \Delta t - 0.5 \|\omega\|_1 \Delta t \quad (\text{III.12})$$

- **Progreso:** recompensa por avance hacia el waypoint activo, recortada a $\pm 0.10\text{m}$ por paso:

$$r_{\text{prog}} = 8.0 \times \text{clip}(d_{t-1} - d_t, -0.10, 0.10) \quad (\text{III.13})$$

- **Bono de recuperación:** $+5.0$ otorgado una sola vez si el dron se estabiliza dentro de los primeros 3s tras la falla (inclinación < 0.20 rad, $\|\boldsymbol{\omega}\| < 1.5$ rad/s, altura > 0.96 m).
- **Recompensas obtenidas al finalizar el episodio:**
 - Waypoint alcanzado: $+10.0$; misión completa: $+100.0$
 - Aterrizaje controlado: $+40.0$
 - Caída (crash): -120.0
 - Tiempo agotado: -30.0

3.2.4.4. Hiperparámetros del agente PPO

El agente se entrena mediante la implementación de PPO de *Stable-Baselines3*, a lo largo de 1 000 000 pasos de entrenamiento. Los hiperparámetros utilizados se presentan en la Tabla 3.9.

3.2.5 Evaluación de controladores

La evaluación de los controladores se realiza en lazo cerrado dentro del entorno *gym-pybullet-drones*, sustituyendo al controlador PID de referencia por los modelos entrenados. Para garantizar la comparabilidad, todos los controladores se evalúan sobre el mismo conjunto de trayectorias reproducibles con coordenadas fijas, definidas en la Tabla 3.10.

Parámetro	Valor
Política	MLP (<i>MlpPolicy</i>)
Pasos totales	1 000 000
Learning rate	3×10^{-4}
n_steps	512
Batch size	128
Épocas por update	10
γ	0.99
λ (GAE)	0.95
Coef. entropía	0.05
Clip range	0.2
Target KL	0.02
$\Delta_{\text{máx}}$	1500 RPM

TABLA 3.9: Hiperparámetros del agente PPO.

En cada evaluación, la secuencia de *waypoints* se genera una única vez mediante la función determinista y se utiliza de forma idéntica a todos los controladores, de modo que las diferencias observadas en las trayectorias registradas se deban al comportamiento de cada modelo.

#	Tipo	Punto A (x, y)	Punto B (x, y)
1	Diagonal larga	(−1.5, −1.5)	(1.5, 1.5)
2	Recto norte	(0.0, −1.5)	(0.0, 1.5)
3	Recto este	(−1.5, 0.0)	(1.5, 0.0)
4	Diagonal inversa	(1.0, −1.0)	(−1.0, 1.0)
5	Diagonal sur	(−1.0, 1.0)	(1.0, −1.0)

TABLA 3.10: Trayectorias de evaluación reproducibles.

La evaluación se realiza en dos escenarios:

- **Escenario nominal (sin falla):**

El dron realiza la trayectoria sin pérdida de potencia de actuadores ($\eta = 1$). Se evalúan tres controladores: **PID** (referencia), **LSTM** y **Mamba**, con el objetivo de determinar qué modelo de imitación reproduce con mayor precisión el comportamiento del controlador PID en condiciones ideales.

- **Escenario de falla:** Se introduce una pérdida de potencia de $p \in \{5\%, 10\%, 15\%\}$ en el motor 0, activada en $t = 3s$ de vuelo. Se evalúan cuatro controladores: **PID**, **LSTM**, **Mamba** y **Mamba+PPO**, con el objetivo de analizar la capacidad de la arquitectura de dos niveles para compensar la falla en comparación con los controladores sin compensación.

En ambos escenarios, cada episodio termina cuando se cumple una de las condiciones descritas en la Tabla 3.11. Los resultados *Llegó* y *Aterrizó* se consideran éxito; *Cayó* y *Tiempo agotado* se consideran falla.

Resultado	Condición	Criterio
Llegó	Éxito	Alcanzó los 8 <i>waypoints</i> con $\ \mathbf{p} - \mathbf{wp}\ < 0.25$ m
Aterrizó	Éxito parcial	$z < 0.05$ m, $ \phi < 20^\circ$, $ \theta < 20^\circ$, $v_z > -0.4$ m/s, $\sum \boldsymbol{\omega} < 1.5$ rad/s
Cayó	Falla	$z < 0.05$ m y ($ \phi > 35^\circ$ o $ \theta > 35^\circ$ o $v_z < -0.5$ m/s)
Tiempo agotado	Falla	Superó los 960 pasos de control (20 s)

TABLA 3.11: Condiciones de terminación de un episodio de evaluación.

Adicionalmente, para obtener resultados estadísticamente más significativos, se realiza una **evaluación estadística** sobre 100 episodios con trayectorias aleatorias bajo el escenario de falla. En cada episodio, los puntos A y B se generan aleatoriamente dentro del espacio de vuelo $[-2.0, 2.0]^2$ m, con una distancia mínima horizontal de $\|A_{xy} - B_{xy}\| \geq 1.0$ m.

La severidad de la falla se evalúa en cada nivel $p \in \{5\%, 10\%, 15\%\}$ utilizando un mismo conjunto de trayectorias, generado con una semilla fija `np.random.seed(42)`, lo que garantiza la reproducibilidad de los resultados.

3.2.6 Métricas de Desempeño

El desempeño de los controladores se cuantifica mediante las siguientes métricas, las cuales se calculan para cada controlador en cada escenario de evaluación.

1. **Tasa de éxito** (τ):

Proporción de trayectorias en las que el dron completa todos los *waypoints*, expresada como porcentaje:

$$\tau = \frac{N_{\text{éxito}}}{N_{\text{total}}} \times 100\% \quad (\text{III.14})$$

2. **Error de posición final** (e_f):

Es la distancia euclidiana tridimensional entre la posición final del dron y el punto destino B al finalizar el episodio:

$$e_f = \|\mathbf{p}_T - \mathbf{p}_B\|_2 \quad [\text{m}] \quad (\text{III.15})$$

Se reportan la media $\bar{e}_f$ y la desviación estándar σ_{e_f} sobre las cinco trayectorias de evaluación.

3. **Error de validación**($\mathcal{L}_{\text{val}}$) :

Es el MSE del modelo sobre el conjunto de validación al finalizar el entrenamiento por imitación:

$$\mathcal{L}_{\text{val}} = \frac{1}{N_{\text{val}}} \sum_{t=1}^{N_{\text{val}}} \|f_{\theta}(X_t) - y_t\|^2 \quad (\text{III.16})$$

Esta métrica aplica únicamente a los modelos LSTM y Mamba, ya que el PID no requiere entrenamiento y el PPO optimiza una función de recompensa distinta.

4. **Número de parámetros entrenables** ($|\theta|$) :

Cuantifica el costo computacional de cada arquitectura y su viabilidad para despliegue en plataformas embebidas.

5. La **latencia de inferencia** (t_{inf}):

Es el tiempo de cómputo promedio de un paso de control (en milisegundos), medido en la plataforma de evaluación. Para que el controlador sea viable en tiempo real a 48 Hz, se requiere $t_{\text{inf}} < 20.8$ ms.

La Tabla 3.12 resume las métricas evaluadas y los controladores a los que aplican.

Métrica	PID	LSTM	Mamba	Mamba+PPO
Tasa de éxito τ (%)	✓	✓	✓	✓
Error final $\bar{e}_f$ (m)	✓	✓	✓	✓
Val loss $\mathcal{L}_{\text{val}}$	—	✓	✓	—
Parámetros $ \theta $	—	✓	✓	✓
Latencia t_{inf} (ms)	✓	✓	✓	✓

TABLA 3.12: Métricas de desempeño y controladores a los que aplican.

CAPÍTULO IV

RESULTADOS PRELIMINARES

Este capítulo presenta los resultados obtenidos durante la implementación del pipeline metodológico propuesto, organizados en tres etapas: (i) análisis de la convergencia del entrenamiento por imitación de los modelos LSTM y Mamba SSM; (ii) evaluación de su desempeño en vuelo nominal dentro del simulador PyBullet; y (iii) análisis de la capacidad de compensación de la arquitectura Mamba+PPO ante fallas de motor con pérdidas de potencia del 5 %, 10 % y 15 %.

Estos resultados permiten validar el enfoque propuesto y establecer una comparación cuantitativa del desempeño de los controladores.

4.1 Convergencia del Entrenamiento por Imitación

La Figura 4.1 muestra las curvas de pérdida de validación de ambos modelos a lo largo de 150 épocas de entrenamiento sobre el dataset de 800 episodios (640 de entrenamiento y 160 de validación). Ambos modelos presentan una convergencia estable: la LSTM alcanza una pérdida mínima de validación de 0.0058, mientras que Mamba SSM obtiene 0.0052, lo que representa una reducción del 10.3 % respecto a la LSTM. Aunque la LSTM converge más rápidamente en las primeras etapas, Mamba logra un valor mínimo inferior hacia el final del entrenamiento, lo que sugiere una mayor capacidad para aproximar las salidas del controlador PID.

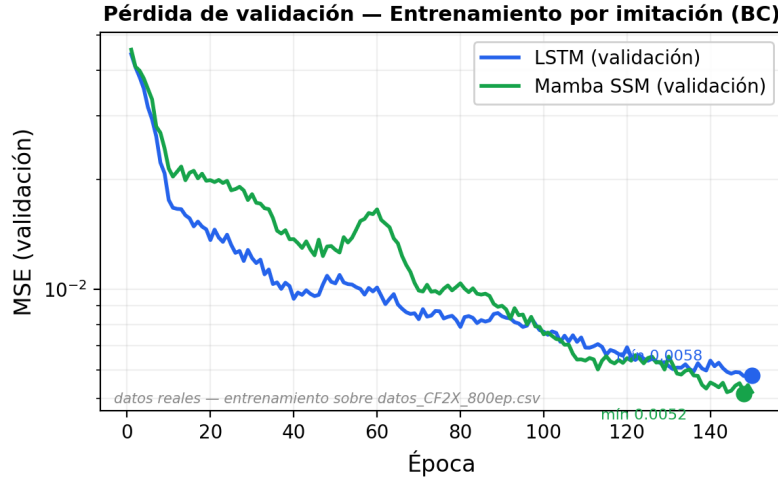


FIGURA 4.1: Curvas de pérdida de validación (MSE) durante el entrenamiento por imitación. LSTM: mín. 0.0058; Mamba SSM: mín. 0.0052.

Tras comprobar la convergencia del entrenamiento por imitación, se evalúa el desempeño de los modelos en el simulador PyBullet, con el fin de comparar su capacidad para reproducir el comportamiento del controlador PID y determinar cuál logra una mejor aproximación.

4.2 Evaluación en Vuelo Normal (Sin Falla de Motor 0)

Para ello, se ejecutan cinco trayectorias representativas sin introducir fallas, con el objetivo de analizar el comportamiento de los modelos en condiciones nominales. Las Figuras 4.2–4.11 muestran las trayectorias 3D obtenidas para los tres controladores: PID (rojo), LSTM (azul) y Mamba SSM (verde), y en la Figura 4.12 se muestra la orientación del dron en cada trayectoria.

1. Trayectorias:

a) Trayectoria 1:

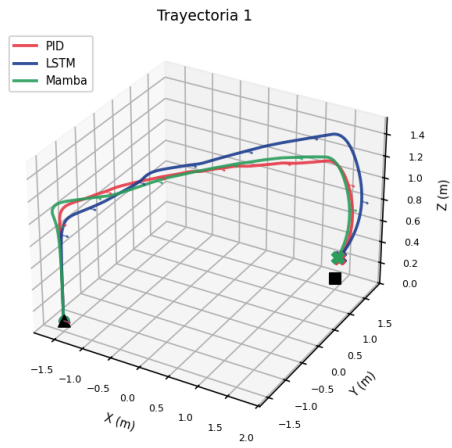


FIGURA 4.2: Seguimiento de trayectoria 3D en vuelo sin fallas

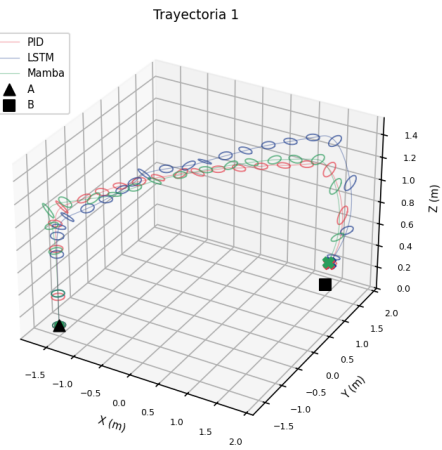


FIGURA 4.3: Orientación 3D del dron

b) Trayectoria 2:

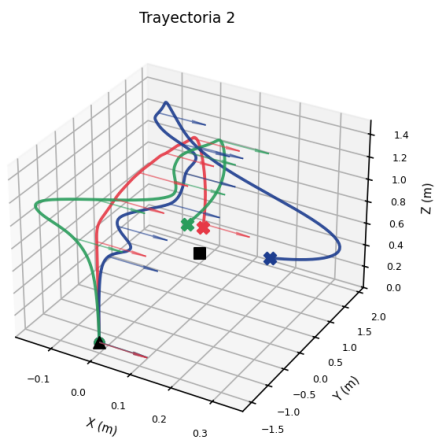


FIGURA 4.4: Seguimiento de trayectoria 3D en vuelo sin fallas

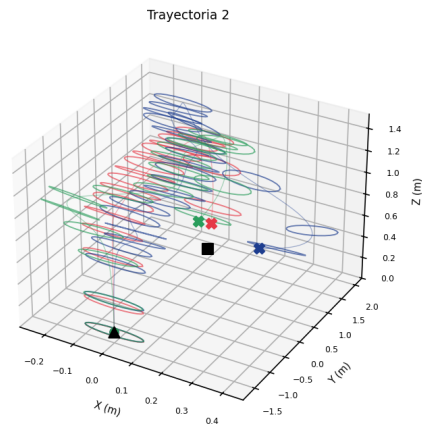


FIGURA 4.5: Orientación 3D del dron

c) **Trayectoria 3:**

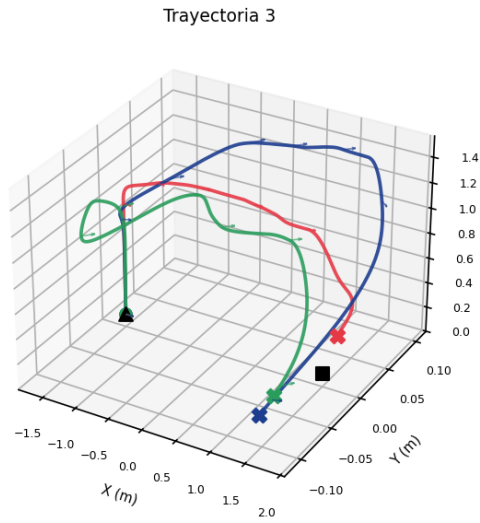


FIGURA 4.6: Seguimiento de trayectoria 3D en vuelo sin fallas

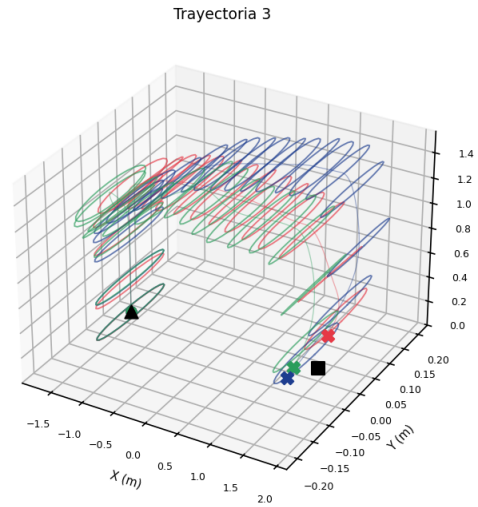


FIGURA 4.7: Orientación 3D del dron

d) **Trayectoria 4:**

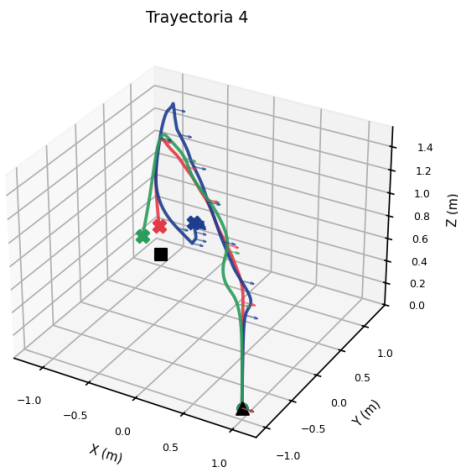


FIGURA 4.8: Seguimiento de trayectoria 3D en vuelo sin fallas

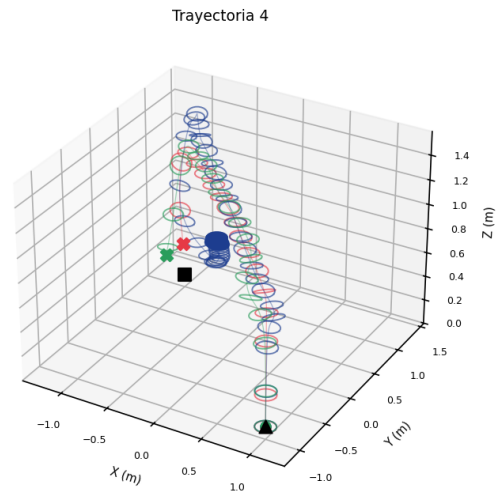


FIGURA 4.9: Orientación 3D del dron

e) **Trayectoria 5:**

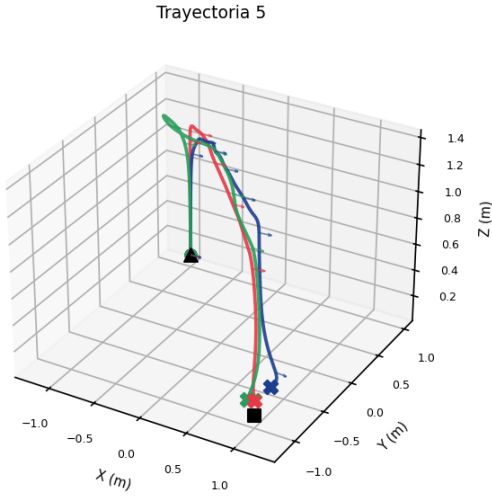


FIGURA 4.10: Seguimiento de trayectoria 3D en vuelo sin fallas

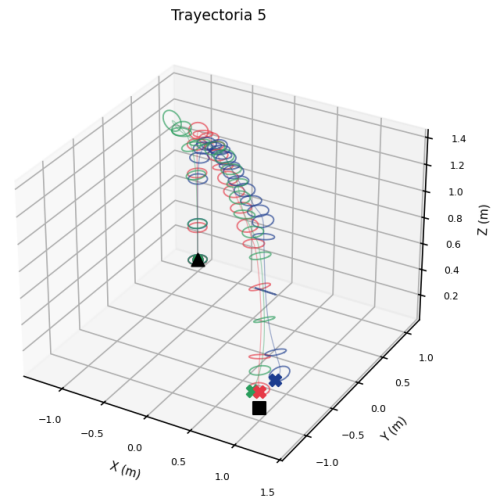


FIGURA 4.11: Orientación 3D del dron

2. Comportamiento de la orientación del dron CF2X en cada trayectoria:

La Figura 4.12 muestra la evolución del Roll, Pitch y Yaw de los tres controladores durante las cinco trayectorias de evaluación en condiciones nominales (sin falla), definidas en la Tabla 3.10.

Estas variables representan la orientación del dron en los tres ejes: *Roll* corresponde a la rotación alrededor del eje longitudinal (inclinación lateral), *Pitch* a la rotación alrededor del eje transversal (inclinación hacia adelante o atrás) y *Yaw* a la rotación alrededor del eje vertical (orientación horizontal).

Las orientaciones parten de cero en todos los casos, lo que corresponde al dron en reposo sobre el suelo.

El Roll y Pitch presentan oscilaciones durante el vuelo, propias de las correcciones del controlador para mantener la trayectoria deseada. En general, LSTM y Mamba SSM replican el comportamiento angular del PID de referencia, aunque con diferencias en la amplitud de las oscilaciones según la trayectoria.

El Yaw se mantiene cercano a cero en la mayoría de los casos, dado que las trayectorias no requieren cambios significativos en la orientación horizontal.

En la Trayectoria 4 se observa que Mamba SSM presenta menor oscilación en Roll y Pitch respecto a LSTM, lo que indica una representación temporal más estable del estado del sistema.

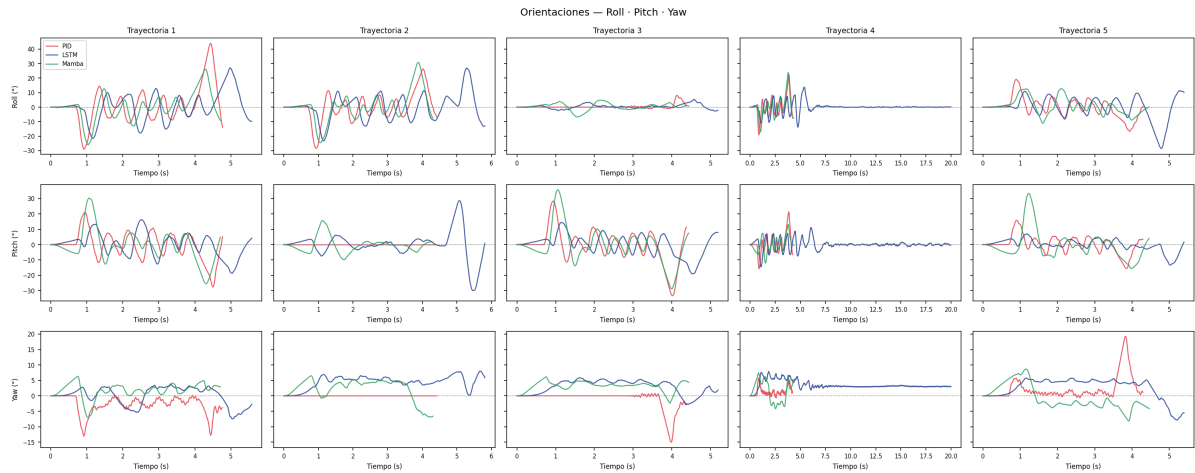


FIGURA 4.12: Orientaciones (Roll - Pitch - Yaw).

Tomando como base las trayectorias y orientaciones presentadas, se evalúa la precisión y la capacidad de seguimiento de los controladores.

En cuatro de las cinco trayectorias evaluadas, los tres controladores logran alcanzar el punto de destino. Sin embargo, en la Trayectoria 2 de las Figuras 4.4 y 4.5, el modelo LSTM no completa la misión: agota el tiempo máximo de simulación (20 s) sin converger al destino, registrando un error final de posición de 0.45m (Figura 4.13).

Por su parte, Mamba SSM mantiene un error final comparable al del PID ($\approx 0.24\text{m}$) en todas las trayectorias, logrando completar el 100 % de las misiones. Además, presenta un seguimiento más preciso de la trayectoria del PID, a diferencia de la LSTM, que muestra mayores desviaciones.

Esto evidencia que Mamba SSM logra una mejor aproximación al comportamiento del controlador de referencia.

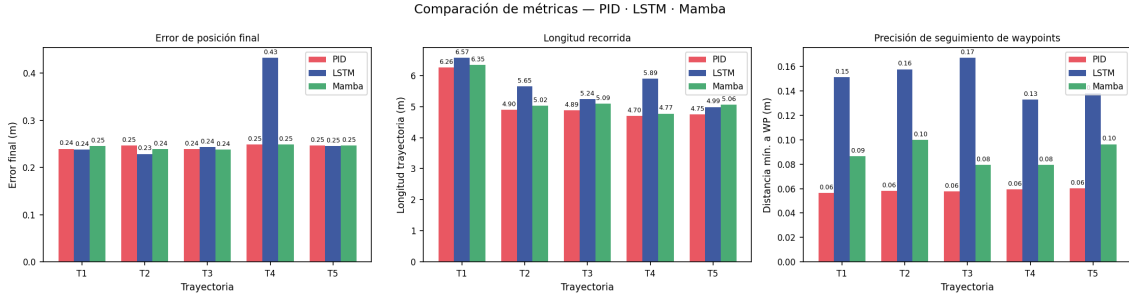


FIGURA 4.13: Comparación de métricas en vuelo normal: error de posición final, longitud de trayectoria recorrida y distancia mínima alcanzada a cada waypoint.

4.3 Evaluación bajo Falla de Motor 0

Se evalúa la respuesta de los cuatro controladores ante la pérdida parcial de potencia en el motor 0, con la falla activada en $t = 3s$ de vuelo. Se consideran 3 niveles de pérdida de potencia del motor 0: 5 %, 10 % y 15 %. La métrica principal es la **tasa de éxito**, definida como la fracción de episodios en los que el dron alcanza el destino o realiza un aterrizaje controlado sin colisionar. La Figura 4.14 resumen los resultados. Se observa que Mamba+PPO es el único controlador que mantiene una tasa de éxito del 100 % hasta una pérdida del 15 %, frente al 20 % del PID y el 19 % de Mamba (BC) en ese mismo nivel de severidad.

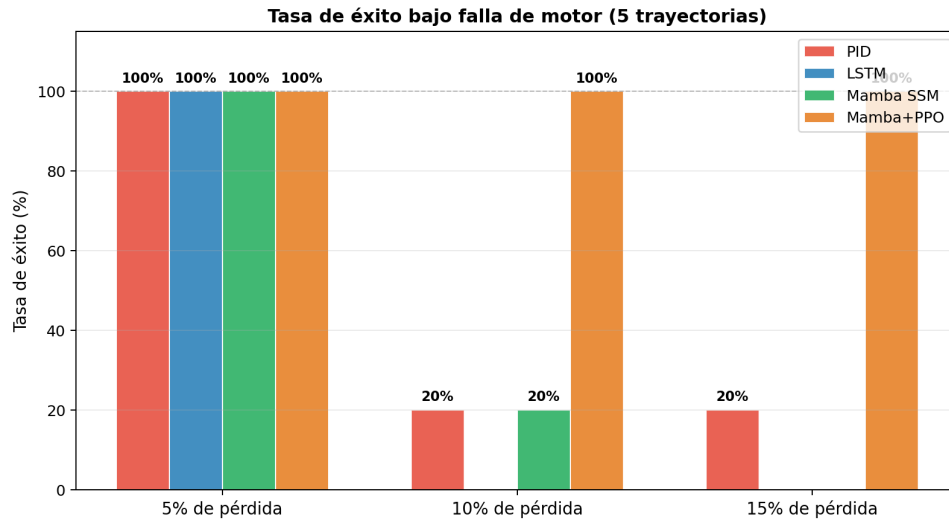


FIGURA 4.14: Tasa de éxito (%) bajo falla del motor 0 para cuatro niveles de pérdida de potencia.

4.3.1 Caso representativo: falla del 15 % de potencia

Las Figuras 4.15–4.19 muestran el comportamiento de los cuatro controladores ante una pérdida del 15% de empuje en el motor 0, activada en $t = 3s$, para cada una de las cinco trayectorias de evaluación definidas en la Tabla 3.10.

■ Trayectoria 1:

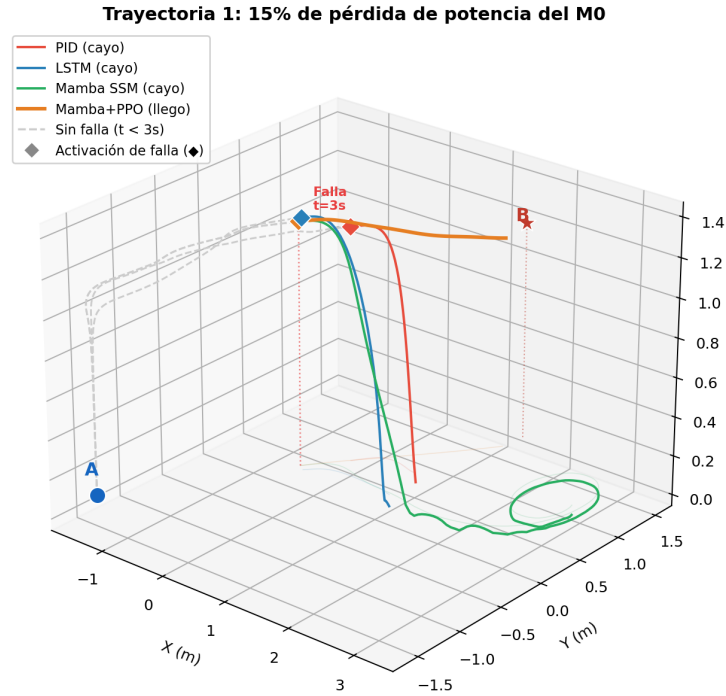


FIGURA 4.15: Trayectoria 1 bajo una pérdida de potencia del 15 %. El controlador PID colisiona aproximadamente en $t = 3.69s$, LSTM en $t = 3.77s$ y Mamba en $t = 6.04s$.

Resultados:

Ante una pérdida del 15 % en el motor 0, el controlador PID pierde sustentación en $t = 3.69s$ y la LSTM en $t = 3.77s$, sin evidenciar capacidad de recuperación. Mamba SSM prolonga el tiempo de vuelo hasta $t = 6.04s$, lo que evidencia una mejor capacidad para mantener la información temporal; sin embargo, finalmente también pierde estabilidad.

En contraste, Mamba+PPO mantiene la estabilidad del sistema y completa la trayectoria hasta el punto B .

■ Trayectoria 2:

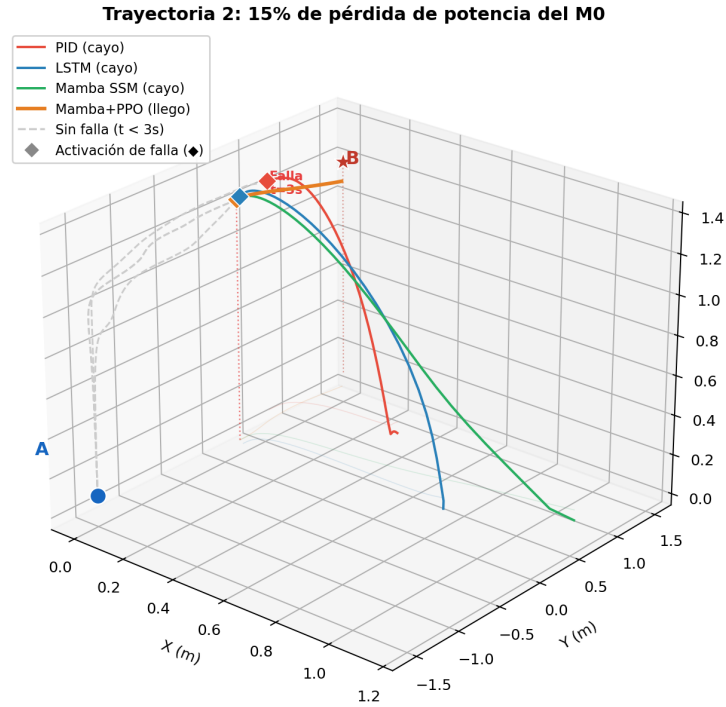


FIGURA 4.16: Trayectoria 2 con una pérdida de potencia del 15 %. Los tres controladores colisionan aproximadamente en $t = 3.8s$ sin recuperar la estabilidad.

Resultados:

Los tres controladores sin compensación colapsan de forma casi simultánea: el PID en $t = 3.81s$, Mamba (BC) en $t = 3.75s$ y la LSTM en $t = 3.73s$, sin mostrar diferencias apreciables entre ellos.

En contraste, Mamba+PPO activa el compensador residual en el instante de la falla y completa la misión sin pérdida de estabilidad.

■ Trayectoria 3:

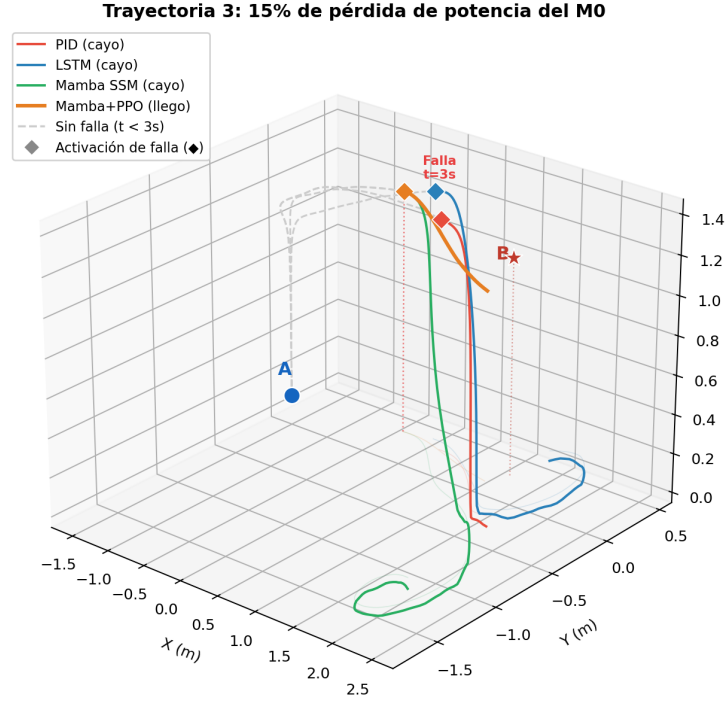


FIGURA 4.17: Trayectoria 3 con una pérdida de potencia del 15 %. El controlador LSTM colisiona en $t = 4.50s$ y Mamba en $t = 5.06s$, registrando tiempos de vuelo superiores a los obtenidos en la trayectoria 2.

Resultados:

En esta trayectoria se observa una mayor resiliencia en los modelos de imitación: la LSTM se mantiene en vuelo hasta $t = 4.50s$ y Mamba SSM hasta $t = 5.06s$ antes de colapsar, mientras que el PID pierde estabilidad en $t = 3.81s$.

A pesar de esta mayor duración, ninguno de los tres logra estabilizarse; Mamba+PPO es el único que completa el recorrido.

■ Trayectoria 4:

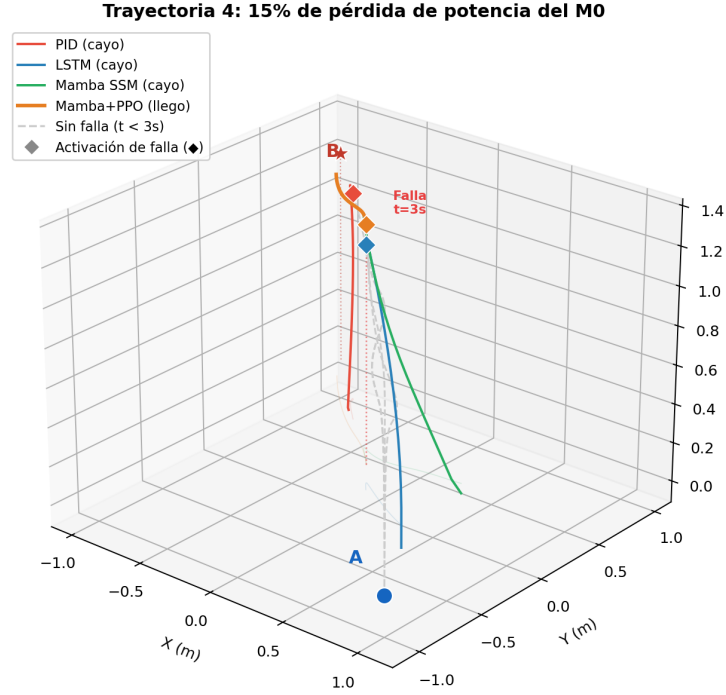


FIGURA 4.18: Trayectoria 4 con una pérdida de potencia del 15 %. Los controladores PID, LSTM y Mamba colisionan aproximadamente en $t = 3.67s$.

Resultados:

Este es el escenario más severo para los controladores sin compensación: PID y LSTM caen simultáneamente en $t = 3.67s$ y Mamba (BC) apenas $0.08s$ después en $t = 3.75s$, reflejando que la geometría de la trayectoria no ofrece margen de recuperación. Solo Mamba+PPO supera la perturbación y alcanza el punto B .

■ Trayectoria 5:

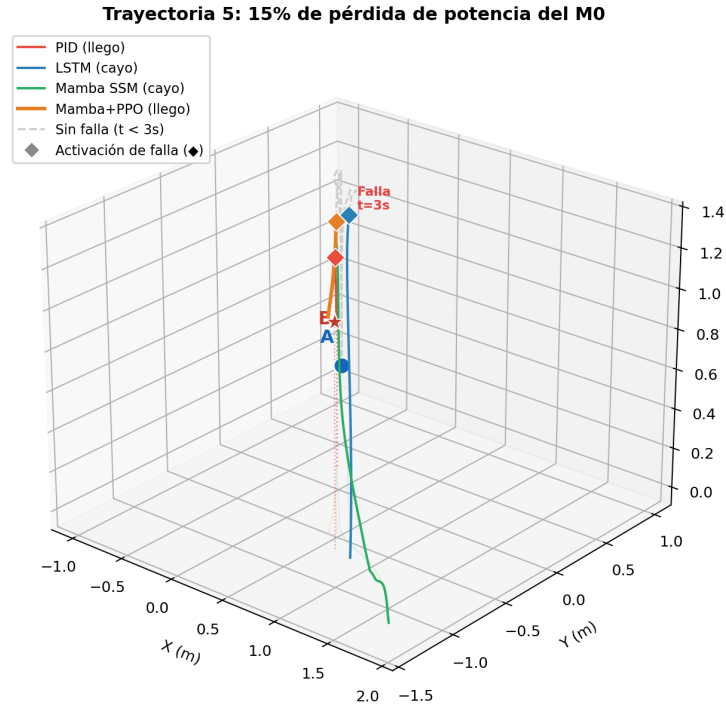


FIGURA 4.19: Trayectoria 5 bajo una pérdida de potencia del 15 %. El controlador LSTM colisiona en $t = 3.67s$ y Mamba en $t = 3.98s$.

Resultados:

Este es el único caso en el que el PID logra completar la misión, lo que evidencia que la combinación de una trayectoria corta y una orientación particular proporciona un mayor margen de tolerancia frente a la falla. No obstante, la LSTM colapsa en $t = 3.67s$ y Mamba SSM en $t = 3.98s$.

En contraste, Mamba+PPO, al igual que el PID, logra completar la trayectoria de forma estable.

Antes de la activación de la falla ($t < 3s$), los cuatro controladores siguen trayectorias prácticamente idénticas (línea gris punteada), lo que confirma que los modelos de imitación replican correctamente el vuelo nominal.

Tras la activación, el comportamiento diverge de forma consistente: el PID y la LSTM pierden sustentación de manera casi inmediata en la mayoría de las trayectorias, mientras que Mamba SSM prolonga su tiempo de vuelo gracias a la inercia de su representación de estado secuencial, aunque tampoco logra recuperar la estabilidad.

En contraste, Mamba+PPO cuyo compensador residual fue entrenado explícitamente bajo escenarios de falla mantiene el control y completa la misión en las cinco trayectorias.

4.4 Síntesis de resultados preliminares

Los experimentos se dividen en tres bloques: entrenamiento por imitación, vuelo nominal y evaluación bajo falla de actuador.

Convergencia del entrenamiento:

Ambas arquitecturas entrenaron durante 150 épocas con Adam ($\alpha = 10^{-3}$), MSE como función de pérdida y reducción automática de la tasa de aprendizaje. La LSTM alcanzó una pérdida de validación mínima de $\mathcal{L}_{\text{val}}^{\text{LSTM}} = 0.0058$; Mamba SSM la redujo a $\mathcal{L}_{\text{val}}^{\text{Mamba}} = 0.0052$, una mejora del 10.3 % que indica una mayor capacidad de aproximación de la política PID a partir de secuencias de estado.

Vuelo nominal:

Evalrados sobre cinco trayectorias de referencia sin falla, Mamba SSM completó el 100 % de las misiones con error de posición final ≤ 0.24 m. La LSTM no completó la Trayectoria 2, registrando un error de 0.45 m como consecuencia de la acumulación de errores por deriva de covariables (*covariate shift*) al operar en lazo cerrado. En conjunto, estos resultados respaldan la selección de Mamba SSM como arquitectura base para la etapa de compensación mediante aprendizaje por refuerzo.

Tolerancia a fallas de actuador:

Se evaluó la arquitectura Mamba+PPO, donde el compensador PPO se entrena sobre el controlador Mamba con parámetros congelados, ante pérdidas de potencia en el motor 0 del 5 %, 10 % y 15 %.

En todos los niveles de severidad, Mamba+PPO completó el 100 % de las misiones con aterrizaje controlado. Como referencia, los controladores sin compensación (PID, LSTM BC y Mamba BC) mantienen el vuelo al 5 %, pero pierden la estabilidad ante pérdidas mayores: al 15 %, el PID cae en $t = 3.7s$ tras la activación de la falla, la LSTM en $t = 3.8s$ y Mamba BC en $t = 6.1s$.

Esta diferencia no corresponde a una comparación directa entre controladores equivalentes dado que los modelos sin compensación carecen del módulo PPO, sino que evidencia la necesidad de un compensador para mantener el control bajo mayores niveles de severidad.

CAPÍTULO V

CONCLUSIONES Y TRABAJOS FUTUROS

En este capítulo se presentan las conclusiones derivadas del desarrollo metodológico y los resultados preliminares alcanzados durante la construcción del pipeline basado en aprendizaje por imitación y aprendizaje por refuerzo para el control tolerante a fallas de un UAV cuadricóptero tipo Crazyflie CF2X. Cabe destacar que la presente entrega esta conformada por las etapas de imitación del controlador PID y compensación de fallas de actuador; la detección autónoma de fallas mediante un estimador de estados constituye el objetivo central de la siguiente entrega. Asimismo, se describen las líneas de trabajo futuras necesarias para completar la validación del sistema y extender su alcance a escenarios de falla más complejos.

5.1 Conclusiones

El presente trabajo propuso y validó preliminarmente un pipeline de control tolerante a fallas para un UAV cuadricóptero tipo Crazyflie CF2X, compuesto por un controlador base aprendido mediante aprendizaje por imitación (BC) y un compensador residual entrenado con PPO. A partir de los resultados obtenidos, se establecen las siguientes conclusiones:

1. Mamba SSM supera a la LSTM en la tarea de imitación:

Ambas arquitecturas convergieron de forma estable a lo largo de las 150 épocas de entrenamiento. Mamba SSM alcanzó una pérdida de validación mínima de $\mathcal{L}_{\text{val}} = 0.0052$, un 10.3 % inferior a la obtenida por la LSTM ($\mathcal{L}_{\text{val}} = 0.0058$), lo

que evidencia una mayor capacidad para aproximar las salidas del controlador PID a partir de secuencias temporales de estado.

2. Mamba SSM presenta mayor robustez en vuelo nominal:

Mamba SSM completó el 100 % de las trayectorias de evaluación en condiciones nominales, mientras que la LSTM no logró completar la Trayectoria 2, registrando un error final de posición de 0.45m (Figura 4.13). Este comportamiento se atribuye a la acumulación de errores por deriva de covariables al operar en lazo cerrado, limitación inherente al Behavioral Cloning que Mamba mitiga gracias a su mecanismo de selección de estado.

3. Los resultados de imitación justifican la elección de Mamba como base del compensador:

La superioridad de Mamba SSM tanto en imitación como en vuelo nominal respalda su selección como arquitectura base para la etapa de compensación mediante aprendizaje por refuerzo. Un controlador base más preciso reduce la carga de corrección exigida al compensador PPO y proporciona una representación de estado más informativa durante el entrenamiento por refuerzo.

4. Mamba+PPO mantiene el dron estable ante fallas de actuador del 15 %:

El compensador PPO, entrenado sobre el modelo Mamba preentrenado con parámetros congelados, aprende a generar acciones residuales que corrigen la política base sin requerir un módulo explícito de detección de fallas. En el caso representativo de la Trayectoria1 con una pérdida de potencia del 15 % en el motor 0, el PID y la LSTM pierden la estabilidad a los 0.69 s y 0.77 s tras la activación de la falla ($t = 3.69$ s y $t = 3.77$ s de simulación), respectivamente, mientras que Mamba BC extiende el vuelo hasta $t = 6.04$ s antes de colapsar. Mamba+PPO ejecuta un aterrizaje controlado, completando la misión.

5. **Mamba+PPO completa todas las trayectorias en todos los niveles de falla evaluados:** En cuatro de las cinco trayectorias evaluadas, ningún controlador sin compensación logra completar la misión ante una pérdida de potencia del 15 %. La única excepción es la Trayectoria 5, que el PID puede completar debido a su menor complejidad geométrica. En contraste, Mamba+PPO completa satisfactoriamente las cinco trayectorias en todos los niveles de pérdida analizados (5 %, 10 % y 15 %), lo que demuestra la efectividad del esquema de compensación propuesto.

5.2 Trabajos Futuros

Como continuación de este trabajo, se identifican las siguientes líneas de investigación:

1. **Incorporación de un estimador de estados para la detección de fallas:**

En el enfoque actual, las fallas son aplicadas directamente en el simulador, sin que el sistema deba identificarlas de forma autónoma. Adicionalmente, el compensador PPO recibe el instante y la severidad de la falla como señales externas hardcodeadas, por lo que permanece inactivo durante el vuelo nominal y solo se activa en el momento programado; en un escenario real, dicho instante es desconocido. Como objetivo inmediato de la siguiente entrega, se plantea el diseño e integración de un estimador de estados basado en filtros de Kalman o en métodos de detección de anomalías que permita identificar la presencia de la falla y estimar su magnitud.

2. **Implementación de LSTM+PPO para comparación:**

El presente trabajo evaluó únicamente la combinación Mamba+PPO. Entrenar un compensador PPO equivalente sobre la política base LSTM permitirá

determinar si la ventaja de Mamba en la fase de imitación se mantiene al incorporar el compensador, o si el PPO compensa las limitaciones de la LSTM hasta equiparar el desempeño. Esta comparación aislará la contribución específica de la arquitectura del controlador base en la tolerancia a fallas final.

3. Extensión a fallas en sensores y perturbaciones externas:

La evaluación realizada se centró en fallas de actuador en el motor0. Como extensión del trabajo, se plantea incorporar escenarios más exigentes que consideren fallas simultáneas en múltiples actuadores, fallas en sensores tales como sesgos o lecturas erróneas en el giroscopio y el acelerómetro, así como perturbaciones aerodinámicas externas. Además, se propone analizar configuraciones en las que múltiples tipos de falla coexistan, lo que constituye un escenario más representativo de aplicaciones reales.

4. Evaluación estadística ampliada:

La evaluación presentada se realizó sobre cinco trayectorias reproducibles y un conjunto acotado de niveles de severidad (5 %, 10 % y 15 %). Como trabajo futuro, se propone ampliar el análisis a un mayor número de episodios aleatorios y a un rango más amplio de severidades, lo que permitirá cuantificar la variabilidad del desempeño y establecer con mayor precisión los límites operacionales de la arquitectura propuesta.

Finalmente, los avances logrados hasta el momento permiten proyectar que la combinación de aprendizaje por imitación y aprendizaje por refuerzo mediante arquitecturas de modelos de espacio de estados constituirá una alternativa eficaz para el control tolerante a fallas en UAVs, con potencial de extensión hacia escenarios de falla más complejos y plataformas físicas reales.

REFERENCIAS BIBLIOGRÁFICAS

- [1] G. K. Furlas and G. C. Karras, “A survey on fault diagnosis and fault-tolerant control methods for unmanned aerial vehicles,” *Machines*, vol. 9, no. 9, p. 197, 2021.
- [2] K. F. Okada, A. S. Morais, L. Ribeiro, C. M. A. da Luz, F. L. Tofoli, G. V. Lima, and L. C. O. Lopes, “Fault-tolerant control for quadcopters under actuator and sensor faults,” *Sensors*, vol. 24, no. 22, p. 7299, 2024.
- [3] D. Kim, J. D. Lee, H. Bang, and J. Bae, “Reinforcement learning-based fault-tolerant control for quadrotor with online transformer adaptation,” 2025.
- [4] A. Freddi, S. Longhi, and A. Monteriù, “A fault-tolerant system for UAVs based on robust control and fault diagnosis,” *Sensors*, vol. 19, no. 21, p. 4721, 2019.
- [5] M. Ozcan and C. Perkgoz, “Deep learning-based proactive fault detection method for enhanced quadrotor safety,” *Aviation*, vol. 28, no. 3, pp. 175–187, 2024.
- [6] Y. Du, P. Huang, Y. Cheng, Y. Fan, and Y. Yuan, “Fault tolerant control of a quadrotor unmanned aerial vehicle based on active disturbance rejection control and two-stage Kalman filter,” *IEEE Access*, vol. 11, pp. 67 556–67 566, 2023.
- [7] X. Liu, Z. Yuan, Z. Gao, and W. Zhang, “Reinforcement learning-based fault-tolerant control for quadrotor UAVs under actuator fault,” *IEEE Transactions on Industrial Informatics*, 2024.

- [8] M. G. Patan and F. Caliskan, “Sensor fault-tolerant control of a quadrotor unmanned aerial vehicle,” *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering*, vol. 235, no. 14, pp. 2171–2184, 2022.
- [9] A. Bekhiti, T. Souanef, H. Toubakh, N. Horri, M. R. Kafi, and Z. Bouzid, “Adaptive UAV control with sensor and actuator faults recovery,” *Aerospace*, vol. 12, no. 3, p. 261, 2025.
- [10] H.-L. Hsu, H. Meng, S. Luo, J. Dong, V. Tarokh, and M. Pajic, “ReForMa: Robust reinforcement learning via adaptive adversary for drones flying under disturbances,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. Yokohama, Japan: IEEE, 2024, pp. 5169–5175.
- [11] National Transportation Safety Board, “Preliminary report: Multirotor drone show incident, lake eola park, orlando, florida,” National Transportation Safety Board (NTSB), Tech. Rep., 2024.
- [12] F. Giral, I. Gómez, R. Vinuesa, and S. Le-Clainche, “Transformer-based fault-tolerant control for fixed-wing UAVs using knowledge distillation and in-context adaptation,” 2024.
- [13] T. van Schijndel and S. Sun, “Fast fault detection on a quadrotor using onboard sensors and a Kalman filter approach,” 2021.
- [14] H. Chen, Z. Song, and X. Zhu, “Disturbance observer-enhanced adaptive fault-tolerant control of a quadrotor UAV against actuator faults and disturbances,” *Drones*, vol. 7, no. 8, p. 541, 2023.
- [15] M. Ahmadi, D. Asadi, S.-Y. Nabavi-Chashmi, and O. Tutsoy, “Active fault-tolerant control of quadrotor UAVs with nonlinear observer-based sliding mode control validated through hardware in the loop experiments,” *Control Engineering Practice*, vol. 137, p. 105557, 2023.

- [16] C. Li, B.-H. Wang, J.-W. Tian, and R.-X. Wang, “Anomaly detection method for UAV sensor data based on LSTM-OCSVM,” *Journal of Chinese Computer Systems*, vol. 42, no. 4, pp. 700–705, 2021.
- [17] F. Fei, Z. Tu, D. Xu, and X. Deng, “Learn-to-recover: Retrofitting UAVs with reinforcement learning-assisted flight control under cyber-physical attacks,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 7358–7364.
- [18] Y. Sohège, M. Quiñones-Grueiro, and G. Provan, “A novel hybrid approach for fault-tolerant control of UAVs based on robust reinforcement learning,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. Xi’an, China: IEEE, 2021, pp. 10 719–10 725.
- [19] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “RMA: Rapid motor adaptation for legged robots,” 2021.
- [20] D. Zhang, A. Loquercio, X. Wu, A. Kumar, J. Malik, and M. W. Mueller, “Learning a single near-hover position controller for vastly different quadcopters,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. London, UK: IEEE, 2023, pp. 1263–1269.
- [21] M. B. Abdelghany, A. M. Moustafa, and M. Moness, “Benchmarking tracking autopilots for quadrotor aerial robotic system using heuristic nonlinear controllers,” *Drones*, vol. 6, no. 12, p. 379, 2022.
- [22] F. Wang, Z. Ma, and H. Gao, “Disturbance observer-based nonsingular fast terminal sliding mode fault tolerant control of a quadrotor UAV with external disturbances and actuator faults,” *International Journal of Control, Automation and Systems*, vol. 20, pp. 1122–1130, 2022.
- [23] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” 2023.

- [24] M. Zare, P. M. Kebria, A. Khosravi, and S. Nahavandi, “A survey of imitation learning: Algorithms, recent developments, and challenges,” 2023.
- [25] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, “Learning to fly—a gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 7512–7519.
- [26] —, “Learning to fly—a gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control,” 2021, iROS 2021. [Online]. Available: <https://github.com/utiasDSL/gym-pybullet-drones>